[TÊN TRƯỜNG]
[TÊN KHOA]

LUẬN VĂN TỐT NGHIỆP

Design and Development of a Multi-Agent Platform for Object-Oriented Enterprise Workflow Automation

(Thiết kế và phát triển nền tảng đa tác tử tự động hoá quy trình doanh nghiệp theo hướng đối tượng)

Sinh viên thực hiện [Họ và tên] – [MSSV]
Giảng viên hướng dẫn [Học hàm, học vị, họ và tên]
Ngành [Tên ngành]

[Thành phố], tháng 12 năm 2026

LỜI CAM ĐOAN

[Sinh viên tự viết và ký: cam đoan luận văn là kết quả nghiên cứu của bản thân dưới sự hướng dẫn của giảng viên hướng dẫn; các số liệu trong luận văn là trung thực và có nguồn gốc rõ ràng.]

LỜI CẢM ƠN

[Sinh viên tự viết.]

TÓM TẮT

Một dự án phần mềm cần nhiều vai trò phối hợp với nhau: phân tích nghiệp vụ, thiết kế kiến trúc, lập kế hoạch, lập trình, kiểm thử và kiểm soát chất lượng. Khi các công đoạn này được thực hiện thủ công, tài liệu phân tích và thiết kế hướng đối tượng (OOAD) thường lệch dần khỏi mã nguồn, và chất lượng sản phẩm thường chỉ được khẳng định mà không được đo. Những năm gần đây, các hệ đa tác tử dựa trên mô hình ngôn ngữ lớn (LLM) đã tự động hoá được một phần chuỗi công việc này. Tuy vậy, bản thân các hệ thống đó hiếm khi được thiết kế theo OOAD, và sản phẩm chúng tạo ra hiếm khi truy vết được từ yêu cầu đến kiểm thử.

Luận văn trình bày AI-CORP, một nền tảng gồm tám tác tử (CEO, BA, Architect, PM, DEV, QC, QA, Marketing) được điều phối bởi một bộ máy quy trình (workflow engine) theo đồ thị có hướng không chu trình. Nền tảng được thiết kế theo kiến trúc phân lớp và thiết kế hướng miền (DDD). Tầng miền độc lập với framework. Mỗi tác tử là một lớp con của `BaseAgent` (mẫu Template Method), kết hợp với hai chiến lược tư duy và hành động được tiêm vào (mẫu Strategy). Vòng đời của tác tử, công việc và quy trình được mô hình hoá bằng máy trạng thái, còn các thành phần giao tiếp với nhau qua sự kiện miền (mẫu Observer). Điểm khác biệt của luận văn là OOAD được áp dụng ở hai mức: cho chính nền tảng và cho mọi dự án do nền tảng sinh ra. Mỗi dự án lần lượt đi qua use case, mô hình miền, sơ đồ UML, mã nguồn phân lớp, kiểm thử chấp nhận theo use case và ma trận truy vết. Sau bước thiết kế có một cổng chất lượng OOAD; mã sinh ra được phân tích tĩnh bằng TypeScript compiler API và được sửa tối đa một vòng trước khi QA duyệt.

Ngoài ra, nền tảng có các tính năng cần cho môi trường doanh nghiệp: phân quyền theo vai trò và theo quyền sở hữu dự án, nhật ký kiểm toán chỉ cho phép ghi thêm, API tích hợp xác thực bằng khoá API, webhook ký HMAC, xuất báo cáo dạng PDF, CSV và JSON, quản lý nhà cung cấp và mô hình LLM với khoá được mã hoá, và điều khiển lượt chạy trong giới hạn ngân sách. Nền tảng cũng phân tích hiệu năng của từng tác tử theo vai trò, cảnh báo khi có bất thường, và dùng mô hình hồi quy ridge để dự báo thời gian và chi phí LLM của từng vai trò; sai số dự báo được đánh giá bằng phương pháp leave-one-out.

Về kết quả, 707 kiểm thử tự động trong 50 bộ kiểm thử đều đạt. Tầng miền đạt độ bao phủ 97,1% số dòng và 88,1% số nhánh. Ba lần chạy đầu-cuối trên PostgreSQL, Redis và nginx lần lượt đạt 24/24, 21/21 và 44/44 kiểm tra. Thời gian xử lý một nút quy

trình, không tính độ trễ của LLM, có trung vị 110 ms và phân vị 99 là 1,73 s. Với 20 người dùng đồng thời, một tiến trình API phục vụ được 604,8 yêu cầu/giây mà không phát sinh lỗi. Quá trình kiểm thử, một đợt rà soát mã phần giao diện và logic, cùng việc cài đặt lại từ đầu đã phát hiện và sửa 50 lỗi, trong đó có một lỗ hổng cho phép người dùng chỉ có quyền xem phê duyệt quy trình qua WebSocket. Hạn chế chính của luận văn là các phép đo dùng một LLM kịch bản để kết quả lặp lại được; vì vậy chất lượng đầu ra khi dùng LLM thật và mức cải thiện nhờ cơ chế học của tác tử chưa được đo định lượng.

Từ khoá: phân tích và thiết kế hướng đối tượng, hệ đa tác tử, mô hình ngôn ngữ lớn, tự động hoá quy trình, mẫu thiết kế, truy vết yêu cầu.

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Nghĩa
ACL	Agent Communication Language
ADR	Architecture Decision Record (bản ghi quyết định kiến trúc)
API	Application Programming Interface
BA	Business Analyst (tác tử phân tích nghiệp vụ)
BDI	Belief–Desire–Intention
BPMN	Business Process Model and Notation
CQRS	Command Query Responsibility Segregation
DAG	Directed Acyclic Graph (đồ thị có hướng không chu trình)
DDD	Domain-Driven Design (thiết kế hướng miền)
DI	Dependency Injection
E2E	End-to-end (đầu-cuối)
FR / NFR	Functional / Non-functional Requirement
GoF	Gang of Four (Gamma, Helm, Johnson, Vlissides)
HMAC	Hash-based Message Authentication Code
JWT	JSON Web Token
LLM	Large Language Model (mô hình ngôn ngữ lớn)
LOO	Leave-one-out (kiểm định chéo bỏ một)
MAPE / MAE	Mean Absolute Percentage Error / Mean Absolute Error
MAS	Multi-Agent System (hệ đa tác tử)
OCP	Open/Closed Principle
OOAD	Object-Oriented Analysis and Design
QA / QC	Quality Assurance / Quality Control
RBAC	Role-Based Access Control
SOLID	SRP, OCP, LSP, ISP, DIP
SPA	Single-Page Application
UC	Use case
UML	Unified Modeling Language

MỤC LỤC

Lời cam đoan

Lời cảm ơn

Tóm tắt

Danh mục từ viết tắt

1 Mở đầu

1.1 Bối cảnh

1.2 Động lực và vấn đề nghiên cứu

1.3 Mục tiêu và phạm vi

1.4 Đóng góp

1.5 Phương pháp nghiên cứu

1.6 Cấu trúc luận văn

2 Cơ sở lý thuyết và tổng quan

2.1 Phân tích và thiết kế hướng đối tượng

2.2 Hệ đa tác tử

2.3 Điều phối quy trình

2.4 Trí tuệ nhân tạo trong kỹ nghệ phần mềm

2.5 Khoảng trống nghiên cứu

3 Phân tích yêu cầu

3.1 Tác nhân và vai trò tác tử

3.2 Yêu cầu chức năng

3.3 Yêu cầu phi chức năng

3.4 Mô hình use case

3.5 Đặc tả use case tiêu biểu

3.6 Mô hình miền

4 Thiết kế hệ thống

4.1 Kiến trúc tổng thể

4.2 Mẫu thiết kế

4.3 Thiết kế tác tử

4.4 Thiết kế bộ máy quy trình

4.5 Thiết kế dữ liệu và lưu trữ

4.6 Áp dụng OOAD cho dự án sinh ra

4.7 Bảo mật và tính năng doanh nghiệp
4.8 Phân tích, dự báo và chất lượng mã
4.9 Thiết kế triển khai

5 Hiện thực

5.1 Công nghệ và quy mô
5.2 Tầng miền
5.3 Tác tử
5.4 Bộ máy quy trình
5.5 Tính năng doanh nghiệp
5.6 Giao diện người dùng
5.7 Cấu hình triển khai

6 Kiểm thử và đánh giá

6.1 Chiến lược kiểm thử
6.2 Kiểm thử tự động và độ bao phủ
6.3 Kiểm thử đầu-cuối
6.4 Thời gian xử lý của một nút quy trình
6.5 Hiệu năng dưới tải
6.6 Độ chính xác của dự báo
6.7 Dự đoán chất lượng mã
6.8 Nghiên cứu tình huống
6.9 Lỗi phát hiện nhờ kiểm thử và rà soát

7 Thảo luận

7.1 RQ1: OOAD có cải thiện kiến trúc của hệ đa tác tử dùng LLM không?
7.2 RQ2: Những mẫu thiết kế nào hiệu quả nhất cho việc điều phối?
7.3 RQ3: Tác tử học từ kết quả như thế nào, và đo điều đó ra sao?
7.4 Mối đe doạ đến tính hợp lệ của đánh giá
7.5 Các hạn chế của nền tảng

8 Kết luận và hướng phát triển

8.1 Kết luận
8.2 Hướng phát triển

Tài liệu tham khảo

Phụ lục A. Danh mục sơ đồ UML

Phụ lục B. Giao diện lập trình (API)

Phụ lục C. Số liệu và cách tái tạo kiểm thử

Kiểm thử tự động

Kiểm thử đầu-cuối

LLM kịch bản

Dữ liệu trình diễn

Phụ lục D. Số liệu hiệu năng

Kiểm thử tải

Thời gian theo bước tác tử

Phụ lục E. Hướng dẫn sử dụng

Cài đặt bằng Docker

Vai trò người dùng

Các thao tác chính

Chương 1
MỞ ĐẦU

1.1 Bối cảnh

Một dự án phần mềm doanh nghiệp, dù có quy mô nhỏ, cũng phải đi qua nhiều công đoạn do nhiều vai trò khác nhau đảm nhận. Người phân tích nghiệp vụ thu thập yêu cầu và viết đặc tả use case. Kiến trúc sư xây dựng mô hình miền, lựa chọn mẫu thiết kế và vẽ các sơ đồ UML. Người quản lý dự án phân chia công việc, lập trình viên hiện thực, người kiểm thử viết và chạy các ca kiểm thử, còn người đảm bảo chất lượng rà soát xem sản phẩm có tuân theo thiết kế và quy trình hay không. Phân tích và thiết kế hướng đối tượng (Object-Oriented Analysis and Design, OOAD) cung cấp một ngôn ngữ chung cho toàn bộ chuỗi công việc này: use case mô tả hành vi của hệ thống từ góc nhìn của tác nhân [3]; mô hình miền và sơ đồ lớp mô tả cấu trúc; sơ đồ tuần tự và sơ đồ trạng thái mô tả hành vi [1], [2]; các mẫu thiết kế ghi lại những lời giải đã được kiểm chứng cho các vấn đề thường gặp [4], [5].

Trong thực tế, chuỗi công việc trên thường đứt gãy ở hai điểm. Thứ nhất, tài liệu thiết kế lệch dần khỏi mã nguồn vì không có cơ chế nào buộc hai bên phải khớp nhau. Thứ hai, chất lượng thường chỉ được khẳng định mà không được đo: một use case được báo là “đã xong” nhưng không có kiểm thử nào chứng minh, hoặc một lớp được cho là “tuân theo kiến trúc phân lớp” nhưng tầng miền lại phụ thuộc vào tầng hạ tầng.

Các mô hình ngôn ngữ lớn (LLM) đã cho thấy khả năng sinh mã nguồn [22] và giải quyết được một phần các tác vụ kỹ nghệ phần mềm thực tế [23]. Các hệ đa tác tử dựa trên LLM như MetaGPT [24], ChatDev [25] hay AutoGen [26] mô phỏng một nhóm phát triển phần mềm, trong đó mỗi vai trò là một tác tử. Tuy nhiên, các hệ thống này chủ yếu đánh giá sản phẩm cuối cùng (mã nguồn có chạy được hay không) và ít quan tâm đến hai câu hỏi mà một luận văn về OOAD cần trả lời: bản thân hệ thống có được thiết kế tốt theo các nguyên lý hướng đối tượng hay không, và sản phẩm do hệ thống tạo ra có truy vết được từ yêu cầu đến kiểm thử hay không.

1.2 Động lực và vấn đề nghiên cứu

Luận văn được xây dựng từ một nền tảng đa tác tử ban đầu gồm năm tác tử (CEO, PM, DEV, QA, Marketing) và một bộ máy quy trình. Qua rà soát, nền tảng này bộc lộ những vấn đề thường gặp ở các hệ thống được phát triển nhanh: tầng miền đã được thiết kế nhưng không được sử dụng; lớp tác tử cơ sở đảm nhận quá nhiều trách nhiệm;

các dịch vụ trộn lẫn logic nghiệp vụ với truy cập dữ liệu; hệ thống không có kho lưu trữ (Repository) hay sự kiện miền; tài liệu UML còn thiếu; và sản phẩm do tác tử tạo ra bị mất sau mỗi lần chạy vì được ghi vào một sandbox không lưu trữ lâu dài.

Xuất phát từ thực trạng đó, luận văn đặt ra ba câu hỏi nghiên cứu:

- **RQ1.** Việc áp dụng có hệ thống các nguyên lý OOAD (kiến trúc phân lớp, SOLID, mẫu thiết kế, thiết kế hướng miền) có cải thiện được kiến trúc của một hệ đa tác tử dùng LLM về tính mô-đun, khả năng mở rộng và khả năng kiểm thử hay không?
- **RQ2.** Những mẫu thiết kế nào phù hợp nhất cho việc điều phối các tác tử và quy trình làm việc của chúng?
- **RQ3.** Các tác tử có thể học từ những lần thực thi trước để cải thiện kết quả theo cách nào, và làm thế nào để đo lường điều đó một cách khách quan?

1.3 Mục tiêu và phạm vi

Mục tiêu của luận văn là thiết kế và hiện thực một nền tảng đa tác tử đáp ứng các yêu cầu sau:

1. Bản thân nền tảng là một ví dụ mẫu về OOAD: có đặc tả use case, mô hình miền, đủ sáu loại sơ đồ UML, kiến trúc phân lớp, các mẫu thiết kế gắn với vấn đề cụ thể và bộ kiểm thử tự động có số đo.
2. Mọi dự án do nền tảng tạo ra cũng tuân thủ OOAD và truy vết được từ use case đến kiểm thử.
3. Chất lượng được đo lường thay vì chỉ được khẳng định, thông qua cổng chất lượng, kiểm thử chấp nhận được thực thi thật, phân tích tĩnh, phân tích hiệu năng tác tử và so sánh dự báo với thực tế.
4. Nền tảng có các tính năng cần thiết cho môi trường doanh nghiệp: phân quyền, nhật ký kiểm toán, API tích hợp, xuất báo cáo và khả năng triển khai.

Phạm vi của luận văn được giới hạn như sau:

- Sản phẩm của nền tảng là một dự án đã được phân tích, thiết kế, hiện thực và kiểm thử; việc triển khai và vận hành dự án đó không thuộc phạm vi. Do vậy, vai trò DevOps của nền tảng ban đầu được loại bỏ, vì vai trò này không tạo ra đầu ra kiểm chứng được và kéo phạm vi sang CI/CD. Thay vào đó, vai trò kiểm soát

chất lượng (QC) được bổ sung, và vai trò đảm bảo chất lượng (QA) được định nghĩa lại theo hướng kiểm soát quy trình (mục 3.1).

- Các dự án sinh ra được viết bằng TypeScript để nền tảng có thể tự phân tích tĩnh và chạy kiểm thử của chúng.
- Các phép đo sử dụng một LLM kịch bản, tức một máy chủ tương thích OpenAI trả lời theo kịch bản định sẵn, để mọi kết quả đều lặp lại được. Nền tảng hoạt động được với bất kỳ cổng LLM tương thích OpenAI nào; việc đánh giá chất lượng đầu ra khi dùng LLM thật nằm ngoài phạm vi đo của luận văn và được trình bày trong phần hạn chế.

1.4 Đóng góp

Luận văn có bốn đóng góp chính:

1. Một kiến trúc tham chiếu cho hệ đa tác tử dùng LLM theo OOAD. Tầng miền (106 tệp) độc lập với framework; mỗi tác tử được xây dựng theo mẫu Template Method kết hợp hai Strategy; tác tử, công việc và quy trình đều có máy trạng thái riêng; các mô-đun giao tiếp qua sự kiện miền; mọi phụ thuộc vào hạ tầng đi qua Repository hoặc cổng (port).
2. Việc áp dụng OOAD cho chính sản phẩm của các tác tử, theo chuỗi từ use case, mô hình miền, sơ đồ UML, mã nguồn phân lớp, kiểm thử chấp nhận đến ma trận truy vết. Chuỗi này được kiểm soát bằng cổng chất lượng OOAD, phân tích tĩnh có phát hiện vi phạm phân lớp, và một vòng tự sửa lỗi có giới hạn chi phí.
3. Một bộ máy quy trình hỗ trợ quy trình con (mẫu Composite), phiên bản bất biến, ảnh chụp và khôi phục trạng thái (mẫu Memento), nhánh song song có ràng buộc phụ thuộc, cùng các mẫu quy trình có kiểu.
4. Một phương pháp đánh giá dựa trên số đo: hiệu năng tác tử được phân tích theo tiêu chí riêng của từng vai trò, mô hình dự báo được đánh giá bằng sai số leave-one-out, phán quyết QA dự đoán được so sánh với phán quyết thực tế, và các lần chạy đầu-cuối đều được lưu vết.

1.5 Phương pháp nghiên cứu

Luận văn sử dụng phương pháp nghiên cứu khoa học thiết kế (design science) [45], gồm ba bước: xây dựng một tạo tác (ở đây là nền tảng), đánh giá tạo tác đó bằng các phép đo lặp lại được, và rút ra tri thức thiết kế. Quá trình phát triển được thực hiện theo vòng lặp: rà soát hiện trạng, phân tích yêu cầu thành các use case, thiết kế, hiện

thực theo hướng kiểm thử trước (TDD) [34], chạy kiểm thử đầu-cuối trên hạ tầng thật (PostgreSQL, Redis, nginx), rà soát mã có hệ thống đối với phần giao diện và logic, và ghi nhận mọi lỗi phát hiện được cùng cách khắc phục. Mỗi số liệu trong luận văn đều có nguồn trong kho mã, dưới dạng tệp kết quả hoặc lệnh để tái tạo.

1.6 Cấu trúc luận văn

Ngoài phần mở đầu, luận văn gồm bảy chương và phần phụ lục. Chương 2 trình bày cơ sở lý thuyết về OOAD, hệ đa tác tử, điều phối quy trình và ứng dụng AI trong kỹ nghệ phần mềm. Chương 3 phân tích yêu cầu và xây dựng mô hình use case. Chương 4 trình bày thiết kế hệ thống, bao gồm kiến trúc, các mẫu thiết kế, thiết kế tác tử, bộ máy quy trình và cách nền tảng áp dụng OOAD cho các dự án sinh ra. Chương 5 mô tả quá trình hiện thực. Chương 6 trình bày kiểm thử và đánh giá. Chương 7 thảo luận câu trả lời cho các câu hỏi nghiên cứu và các hạn chế. Chương 8 kết luận và nêu hướng phát triển. Phần phụ lục gồm danh mục sơ đồ UML, tài liệu API, số liệu kiểm thử, số liệu hiệu năng và hướng dẫn sử dụng.

Chương 2
CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN

Chương này trình bày bốn nhóm kiến thức làm cơ sở cho nền tảng: phân tích và thiết kế hướng đối tượng, hệ đa tác tử, điều phối quy trình và ứng dụng trí tuệ nhân tạo trong kỹ nghệ phần mềm. Phần cuối chương chỉ ra khoảng trống nghiên cứu mà luận văn hướng tới.

2.1 Phân tích và thiết kế hướng đối tượng

2.1.1 Mô hình hoá bằng use case và UML

Jacobson đề xuất phương pháp phát triển phần mềm hướng use case [3], trong đó mỗi use case là một chuỗi tương tác giữa một tác nhân (actor) và hệ thống, mang lại một kết quả có giá trị cho tác nhân đó. Use case vừa là đơn vị yêu cầu, vừa là đơn vị kiểm thử chấp nhận, đồng thời là trục để truy vết đến thiết kế và mã nguồn. Các quan hệ `«include»` (hành vi con bắt buộc), `«extend»` (hành vi mở rộng có điều kiện) và tổng quát hoá giúp tránh lặp lại đặc tả.

Ngôn ngữ mô hình hoá thống nhất UML [1], [2] hợp nhất ký pháp của Booch, Rumbaugh và Jacobson. UML 2.5 chia các sơ đồ thành hai nhóm: nhóm cấu trúc (lớp, đối tượng, thành phần, triển khai, gói, cấu trúc hỗn hợp) và nhóm hành vi (use case, tuần tự, hoạt động, máy trạng thái, giao tiếp, thời gian, tổng quan tương tác). Kruchten tổ chức tài liệu kiến trúc theo mô hình 4+1 góc nhìn (logic, tiến trình, phát triển, vật lý, cộng thêm kịch bản) [12]; luận văn sử dụng cách tổ chức tương tự với sáu nhóm sơ đồ: use case, lớp, tuần tự, hoạt động, máy trạng thái và kiến trúc (gói, thành phần, triển khai).

Larman mô tả một quy trình OOAD lặp gồm các bước: xác định use case; xây dựng mô hình miền (khái niệm, thuộc tính, liên kết, chưa có phương thức); vẽ sơ đồ tuần tự hệ thống; viết hợp đồng thao tác; và thiết kế lớp theo các nguyên tắc gán trách nhiệm GRASP (Information Expert, Creator, Controller, Low Coupling, High Cohesion, v.v.) [9]. Nền tảng áp dụng chính chuỗi bước này cho các dự án sinh ra: tác tử BA viết use case, còn tác tử Architect xây dựng mô hình miền, sơ đồ lớp, sơ đồ tuần tự và lựa chọn mẫu thiết kế.

2.1.2 Các nguyên lý SOLID và mô-đun hoá

Parnas chỉ ra rằng tiêu chí đúng đắn để phân rã mô-đun là che giấu những quyết định thiết kế dễ thay đổi [55]. Martin hệ thống hoá năm nguyên lý thiết kế lớp, gọi tắt là SOLID [7], [8]:

- **SRP** (Single Responsibility): một lớp chỉ có một lý do để thay đổi.
- **OCP** (Open/Closed), bắt nguồn từ Meyer [10]: mở để mở rộng, đóng để sửa đổi.
- **LSP** (Liskov Substitution) [11]: lớp con thay thế được lớp cha mà không phá vỡ hành vi mong đợi.
- **ISP** (Interface Segregation): nhiều giao diện nhỏ chuyên biệt tốt hơn một giao diện lớn.
- **DIP** (Dependency Inversion): mô-đun cấp cao và cấp thấp đều phụ thuộc vào trừu tượng.

Kiến trúc sạch (Clean Architecture) của Martin [8] và kiến trúc phân lớp của Evans [6] cùng đưa ra quy tắc phụ thuộc (dependency rule): mã ở vòng trong (tầng miền) không được biết đến các vòng ngoài (ứng dụng, hạ tầng, giao diện). Nền tảng áp dụng quy tắc này ở hai nơi: cho mã nguồn của chính nền tảng, và như một luật được kiểm tra tự động trên mã do tác tử DEV sinh ra (mục 4.6).

2.1.3 Mẫu thiết kế

Gamma và cộng sự (nhóm GoF) phân loại 23 mẫu thiết kế thành ba nhóm: khởi tạo (Factory Method, Abstract Factory, Builder, Prototype, Singleton), cấu trúc (Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy) và hành vi (Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor) [4]. Fowler bổ sung các mẫu dành cho ứng dụng doanh nghiệp như Repository, Data Mapper, Service Layer, Unit of Work và Domain Model [5]. Evans đưa ra các khối xây dựng của thiết kế hướng miền: Entity, Value Object, Aggregate, Domain Event, Repository, Factory, Domain Service [6].

Một mẫu thiết kế chỉ có giá trị khi được gắn với một vấn đề cụ thể trong ngữ cảnh. Vì vậy, luận văn trình bày các mẫu thiết kế dưới dạng bảng gồm ba cột: vấn đề, mẫu được chọn và lớp hiện thực (mục 4.2).

2.1.4 Truy vết yêu cầu

Gotel và Finkelstein định nghĩa truy vết yêu cầu là khả năng theo dõi vòng đời của một yêu cầu theo cả hai chiều, từ nguồn gốc đến đặc tả, thiết kế, hiện thực và kiểm thử

[44]. Chuẩn IEEE 830 coi tính truy vết được là một thuộc tính của một bản đặc tả yêu cầu tốt [42]. Ma trận truy vết là công cụ phổ biến nhất cho mục đích này; giá trị của ma trận phụ thuộc vào việc các ô được cập nhật từ dữ liệu thực (kiểm thử đã chạy) hay chỉ được điền thủ công. Nền tảng xây dựng ma trận truy vết từ kết quả thực thi kiểm thử chấp nhận (mục 4.6).

2.1.5 Chất lượng phần mềm: QA và QC

Bộ tiêu chuẩn ISO 9000 phân biệt đảm bảo chất lượng (QA), tức tạo niềm tin rằng yêu cầu chất lượng sẽ được đáp ứng, với kiểm soát chất lượng (QC), tức trực tiếp đáp ứng yêu cầu chất lượng [53]. QA hướng vào quy trình và mang tính phòng ngừa, còn QC hướng vào sản phẩm và mang tính phát hiện. Tiêu chuẩn ISO/IEC 25010 định nghĩa mô hình chất lượng sản phẩm với các đặc tính như tính phù hợp chức năng, hiệu năng, bảo mật, khả năng bảo trì (gồm tính mô-đun, khả năng tái sử dụng, khả năng phân tích, sửa đổi và kiểm thử) [43]. Sự phân biệt giữa QA và QC là cơ sở để nền tảng tách thành hai tác tử với trách nhiệm khác nhau (mục 3.1).

2.2 Hệ đa tác tử

2.2.1 Tác tử và kiến trúc tác tử

Wooldridge định nghĩa tác tử là một hệ thống máy tính đặt trong một môi trường, có khả năng hành động tự chủ để đạt được mục tiêu; tác tử thông minh có tính phản ứng, tính chủ động và năng lực xã hội [14]. Kiến trúc BDI (Belief-Desire-Intention) của Rao và Georgeff mô hình hoá tác tử thông qua niềm tin về thế giới, mong muốn và ý định đã cam kết [15]. Chu trình nhận thức, suy nghĩ và hành động của BDI tương ứng trực tiếp với cặp phương thức `think()` / `act()` của tác tử trong nền tảng, trong đó bộ nhớ dài hạn đóng vai trò niềm tin và mục tiêu của công việc đóng vai trò mong muốn.

2.2.2 Giao tiếp và phối hợp

FIPA chuẩn hoá ngôn ngữ giao tiếp tác tử (ACL) với các hành động ngôn từ (performative) như `inform`, `request`, `propose`, `accept-proposal`, `reject-proposal` [16]. Giao thức Contract Net của Smith là mô hình kinh điển cho việc phân công: người quản lý gọi thầu, các tác tử gửi đề xuất, sau đó người quản lý trao hợp đồng [17]. Stone và Veloso phân loại hệ đa tác tử theo mức độ đồng nhất và mức độ giao tiếp giữa các tác tử [18].

Nền tảng lựa chọn cách phối hợp có điều phối trung tâm. Bộ máy quy trình quyết định tác tử nào thực hiện việc gì theo một đồ thị đã định nghĩa trước, còn các tác tử trao đổi

thông điệp có kiểu (INFORM, REQUEST, PROPOSE, ACCEPT, REJECT) qua một bus thông điệp (mẫu Mediator) trong các phiên cộng tác như lập trình cặp. Lựa chọn này chấp nhận giảm tính tự chủ của tác tử để đổi lấy tính dự đoán được và khả năng truy vết, phù hợp với một quy trình phát triển phần mềm có cổng chất lượng.

2.3 Điều phối quy trình

BPMN 2.0 là chuẩn ký pháp cho quy trình nghiệp vụ, gồm các phần tử sự kiện, công việc và cổng (gateway) rẽ nhánh hoặc song song [19]. Mạng Petri cung cấp nền tảng hình thức để phân tích tính đạt được, tính sống và tính bị chặn của quy trình [20]. Van der Aalst và cộng sự tổng hợp các mẫu quy trình (workflow patterns) như tuần tự, tách song song, đồng bộ, lựa chọn độc quyền, vòng lặp tuỳ ý và quy trình con [21]. Statecharts của Harel mở rộng máy trạng thái hữu hạn với trạng thái lồng nhau và trạng thái song song [33], và là cơ sở cho sơ đồ máy trạng thái của UML.

Bộ máy quy trình của nền tảng hiện thực một tập con các mẫu cần thiết cho phát triển phần mềm: tuần tự, lựa chọn có điều kiện (theo phán quyết của QA), vòng lặp (khi QA từ chối, công việc quay lại DEV), tách và đồng bộ song song có phụ thuộc, quy trình con, và điểm dừng chờ con người phê duyệt. Đồ thị được kiểm tra không chu trình bằng sắp xếp tô-pô, trừ các cạnh quay lui được khai báo tường minh.

2.4 Trí tuệ nhân tạo trong kỹ nghệ phần mềm

2.4.1 Mô hình ngôn ngữ lớn và sinh mã

Kiến trúc Transformer [32] là cơ sở của các LLM hiện nay. Chen và cộng sự giới thiệu Codex và bộ đánh giá HumanEval cho bài toán sinh hàm từ mô tả [22]. SWE-bench đo khả năng giải quyết các issue thực tế trên GitHub và cho thấy khoảng cách lớn giữa việc sinh một hàm và việc sửa lỗi trong cả một kho mã [23]. Kỹ thuật gợi ý chuỗi suy luận (chain-of-thought) cải thiện khả năng suy luận nhiều bước [30]; ReAct xen kẽ các bước suy luận với hành động gọi công cụ [27]; còn sinh tăng cường truy xuất (RAG) đưa tri thức bên ngoài vào ngữ cảnh của mô hình [31].

2.4.2 Tác tử LLM và hệ đa tác tử cho phát triển phần mềm

MetaGPT mã hoá quy trình vận hành chuẩn (SOP) của một công ty phần mềm thành các vai trò (quản lý sản phẩm, kiến trúc sư, kỹ sư, v.v.) trao đổi tài liệu có cấu trúc với nhau [24]. ChatDev chia quá trình phát triển thành các giai đoạn thiết kế, lập trình và kiểm thử, trong đó mỗi giai đoạn là một cuộc hội thoại giữa hai vai trò [25]. AutoGen

cung cấp một khung lập trình cho hội thoại giữa nhiều tác tử [26]. Về học từ kinh nghiệm, Reflexion cho phép tác tử tự viết nhận xét về các lần thất bại và đưa nhận xét đó vào lần thử tiếp theo [28]; Generative Agents lưu trữ ký ức, truy xuất theo mức độ liên quan và tổng hợp thành các suy ngẫm [29].

Cơ chế học của nền tảng gần với Reflexion và Generative Agents. Mỗi kết quả của tác tử được đánh giá bằng một tín hiệu khách quan (cổng OOAD, kết quả kiểm thử của QC, phán quyết của QA); định kỳ, các kết quả này được chắt lọc thành bài học, lưu trong bộ nhớ của vai trò tương ứng và đưa vào lời nhắc ở những lần sau (mục 4.3.4).

2.4.3 Đo chất lượng mã và ước lượng công sức

McCabe định nghĩa độ phức tạp chu trình (cyclomatic complexity) là số đường đi độc lập qua đồ thị luồng điều khiển [35]. Halstead đo thể tích chương trình dựa trên số toán tử và toán hạng [36]. Oman, Hagemeister và Coleman kết hợp hai độ đo này với số dòng mã để tạo thành chỉ số bảo trì (Maintainability Index) [37], [38]. Về ước lượng công sức, COCOMO của Boehm là mô hình hồi quy kinh điển giữa kích thước và công sức [39]. Hồi quy ridge [40] bổ sung số hạng phạt chuẩn L2 để mô hình ổn định hơn khi dữ liệu ít và các biến có tương quan với nhau. Phương pháp kiểm định chéo bỏ một (leave-one-out) cho ước lượng sai số tổng quát ít thiên lệch khi kích thước mẫu nhỏ [46].

2.5 Khoảng trống nghiên cứu

Các hệ đa tác tử cho phát triển phần mềm hiện có [24]–[26] tập trung vào việc sinh ra mã nguồn chạy được. Có ba khía cạnh còn ít được đề cập:

1. Bản thân các hệ thống này hiếm khi được trình bày như một sản phẩm OOAD có mô hình miền, có mẫu thiết kế gắn với vấn đề cụ thể và có kiểm thử với độ bao phủ đo được.
2. Sản phẩm do tác tử tạo ra thường thiếu chuỗi truy vết từ use case qua thiết kế đến kiểm thử đã thực thi, và thiếu một cổng kiểm tra xem mã nguồn có tuân thủ kiến trúc đã thiết kế hay không.
3. Việc đánh giá thường dựa trên tỉ lệ thành công tổng thể, ít khi đo riêng từng vai trò theo tiêu chí phù hợp với vai trò đó, và ít khi so sánh dự báo với kết quả thực tế.

Luận văn giải quyết cả ba khía cạnh trên, trong giới hạn đã nêu ở mục 1.3 là các phép đo sử dụng LLM kịch bản để bảo đảm tính lặp lại.

Chương 3
PHÂN TÍCH YÊU CẦU

Chương này xác định những đối tượng sử dụng nền tảng, các chức năng nền tảng phải cung cấp và các yêu cầu chất lượng nền tảng phải đạt. Đặc tả đầy đủ của 31 use case nằm trong tệp `docs/requirements/use-cases.md`; ma trận truy vết từ yêu cầu đến use case, thiết kế, mã nguồn và kiểm thử nằm trong tệp `docs/requirements/traceability-matrix.md`. Nội dung chương tóm tắt các tài liệu này và phân tích những quyết định chính.

3.1 Tác nhân và vai trò tác tử

Nền tảng có hai loại tác nhân là con người và tác tử AI.

Tác nhân con người. Nền tảng phân quyền theo vai trò (RBAC) [47] với ba vai trò như trong bảng sau.

Vai trò	Quyền chính
Quản trị viên (admin)	Quản lý người dùng và vai trò, cấu hình mô hình LLM cho tác tử, xem nhật ký kiểm toán, quản lý khoá API và webhook của mọi người, theo dõi sức khoẻ hệ thống
Thành viên hội đồng quản trị (board member)	Tạo và sở hữu dự án, duyệt hoặc từ chối các bước cần con người, xem chi phí, báo cáo và phân tích, xuất dữ liệu, quản lý khoá API và webhook của mình
Người xem (viewer)	Chỉ đọc: xem dự án, báo cáo và tài liệu sinh ra

Bên cạnh vai trò, mỗi dự án có một chủ sở hữu. Thành viên hội đồng quản trị chỉ xử lý được các yêu cầu phê duyệt được giao cho mình và chỉ chỉnh sửa được dự án của mình. Tác tử AI không phải là người dùng đăng nhập; mọi hành động của tác tử được ghi vào nhật ký kiểm toán với loại tác nhân `agent`.

Tác tử AI. Nền tảng ban đầu có năm tác tử. Việc rà soát lại các vai trò dẫn đến ba thay đổi sau:

1. Bổ sung hai tác tử BA và Architect, vì một quy trình OOAD cần có người viết use case và người xây dựng mô hình miền trước khi lập kế hoạch.
2. Loại bỏ tác tử DevOps. Sản phẩm của nền tảng là một dự án đã được phân tích, thiết kế, hiện thực và kiểm thử; việc triển khai dự án đó không thuộc phạm vi. Trong nền tảng ban đầu, tác tử DevOps chưa từng được đưa vào quy trình, chỉ

sinh ra văn bản không được sử dụng, và sandbox cũng không có mạng hay ổ lưu trữ để triển khai. Nếu giữ lại, tác tử này sẽ là một vai trò không có đầu ra kiểm chứng được và kéo phạm vi đề tài sang CI/CD.

3. Tách riêng QA và QC theo định nghĩa của ISO 9000 [53]. Tác tử QA cũ đảm nhận cả hai nhiệm vụ và trên thực tế không hoàn thành tốt nhiệm vụ nào.

Tác tử	Trách nhiệm	Đầu ra kiểm chứng được
CEO	Định hướng, chia mốc, đóng dự án	Mốc (milestone), báo cáo đóng dự án
BA	Phân tích yêu cầu	`docs/use-cases.md` (use case có ID, tác nhân, luồng chính, luồng thay thế, tiêu chí chấp nhận)
Architect	Thiết kế	Mô hình miền, sơ đồ use case, lớp, tuần tự (PlantUML), `docs/architecture-patterns.md`, khung mã phân lớp
PM	Lập kế hoạch	Một công việc cho mỗi use case đã duyệt, có ước lượng chi phí
DEV	Hiện thực	Mã TypeScript theo bốn tầng, chú thích `UC-xxx`
QC (sản phẩm)	Kiểm soát chất lượng	`tests/UC-xxx.spec.ts`, kết quả chạy Jest, `docs/traceability-matrix.md`, độ bao phủ use case
QA (quy trình)	Đảm bảo chất lượng	Cổng OOAD sau thiết kế, rà soát mã so với thiết kế, phán quyết PASS/FAIL điều khiển vòng lặp về DEV
Marketing	Truyền thông	Thông báo sản phẩm

3.2 Yêu cầu chức năng

Các yêu cầu chức năng được chia thành bốn nhóm lớn (mã trong ngoặc là mã nhóm trong ma trận truy vết).

Hệ thống tác tử (FR1). Nhóm này gồm tám vai trò tác tử nêu trên; khả năng học từ các kết quả trước thông qua bộ nhớ đã được chắt lọc; khả năng cộng tác theo các mẫu lập trình cặp và chuỗi rà soát; và khả năng giao việc con cho nhau qua bus thông điệp.

Bộ máy quy trình (FR2). Nhóm này gồm quy trình con và lồng ghép quy trình; phiên bản bất biến và khôi phục về ảnh chụp trạng thái; thực thi song song có giải quyết phụ thuộc; và các mẫu quy trình có tham số định kiểu.

Tính năng doanh nghiệp (FR3). Nhóm này gồm RBAC với quyền chi tiết và quyền sở hữu dự án; nhật ký kiểm toán đầy đủ; API tích hợp cho công cụ bên ngoài (khoá API, webhook); xuất dữ liệu và báo cáo; quản lý nhà cung cấp và mô hình LLM ngay

trong ứng dụng, với khoá API được mã hoá; và điều khiển lượt chạy (tạm dừng, tiếp tục, huỷ) có ràng buộc về ngân sách và phê duyệt.

Tính năng AI/ML (FR4). Nhóm này gồm phân tích hiệu năng và quá trình học của tác tử; mô hình ước lượng dựa trên dữ liệu lịch sử; dự đoán chất lượng mã bằng phân tích tĩnh; và gợi ý tối ưu dựa trên các bất thường phát hiện được.

Ma trận truy vết có tổng cộng 124 yêu cầu chức năng và phi chức năng, trong đó 115 yêu cầu ở trạng thái *Implemented*, 1 yêu cầu ở trạng thái *Partial* và 8 yêu cầu ở trạng thái *Planned*. Các yêu cầu chưa thực hiện được giữ lại trong ma trận và ghi rõ trạng thái. Mỗi yêu cầu đã hiện thực đều trỏ đến tệp mã nguồn và tệp kiểm thử tương ứng.

3.3 Yêu cầu phi chức năng

Nhóm	Yêu cầu	Cách kiểm chứng
Kiến trúc	Tuân thủ SOLID; mẫu thiết kế được áp dụng có lý do; tách tầng miền, ứng dụng, hạ tầng; tiêm phụ thuộc xuyên suốt	Rà soát thiết kế (chương 4), luật phân lớp được kiểm tra tự động trên mã sinh ra
Kiểm thử	Tầng miền ≥ 90% hàm, dòng, lệnh và ≥ 85% nhánh; toàn API có ngưỡng sàn; kiểm thử đầu-cuối cho các hành trình chính	Ngưỡng `coverageThreshold` của Jest làm hỏng lệnh `test:cov` nếu không đạt; các lần chạy đầu-cuối có lưu vết
Hiệu năng	Mỗi nút quy trình ≤ 2 s (không tính độ trễ LLM); các đọc trên bảng điều khiển trong ngân sách p50/p95/p99 với 20 người dùng đồng thời, lỗi ≤ 1%	Đo bước quy trình (mục 6.4), kiểm thử tải có ngân sách (mục 6.5)

Nhóm	Yêu cầu	Cách kiểm chứng
Bảo mật	Băm mật khẩu bcrypt; JWT có hạn (mặc định 8 giờ); tự đăng ký chỉ nhận quyền xem; khoá của nhà cung cấp LLM được mã hoá và không bao giờ trả về; khoá API chỉ mang phạm vi mà vai trò chủ sở hữu cho phép; nhật ký kiểm toán không lưu bí mật; webhook ký HMAC và chặn địa chỉ mạng nội bộ; sandbox cách ly; cấu hình production sai thì không khởi động	Kiểm thử đơn vị và đầu-cuối (mục 6.3)
Tin cậy	Thử lại có giãn cách mũ; khôi phục quy trình khi khởi động lại; kiểm tra sức khoẻ cho cân bằng tải	Kiểm thử bộ máy quy trình, kiểm thử sức khoẻ
Tài liệu	Đủ sơ đồ use case, lớp, tuần tự, hoạt động, máy trạng thái, thành phần và triển khai	39 sơ đồ PlantUML được render tự động, lệnh render thất bại nếu có lỗi cú pháp

3.4 Mô hình use case

Nền tảng có 31 use case đã được đặc tả, chia thành bốn nhóm như trong bảng sau.

Nhóm	Use case
Quản lý dự án (con người)	UC-001 Xác thực · UC-002 Tạo dự án · UC-005 Duyệt bước quy trình · UC-006 Từ chối bước quy trình · UC-007 Theo dõi tiến độ · UC-008 Xem chi phí · UC-020 Quản lý Kanban · UC-022 Tạo báo cáo dự án · UC-026 Duyệt sản phẩm của dự án
Tác tử	UC-009 Phân tích yêu cầu (BA) · UC-010 Thiết kế kiến trúc (Architect) · UC-011 Lập kế hoạch (PM) · UC-012 Thực hiện công việc phát triển (DEV) · UC-013 Rà soát mã (QA) · UC-014 Chiến lược marketing · UC-015 Điều phối quy trình (CEO) · UC-016 Kiểm chứng hiện thực theo use case (QC) · UC-023 Phân tích chất lượng mã (DEV, QA)
Hệ thống	UC-004 Thực thi quy trình · UC-017 Quản lý bộ nhớ tác tử · UC-018 Định tuyến yêu cầu LLM · UC-019 Khôi phục quy trình · UC-021 Phòng họp tác tử · UC-024 Dự báo công sức và chi phí

Nhóm	Use case
Quản trị và tích hợp	UC-003 Cấu hình tác tử · UC-025 Theo dõi sức khoẻ hệ thống · UC-029 Quản lý người dùng và vai trò · UC-030 Xem nhật ký kiểm toán · UC-031 Quản lý khoá API và webhook · UC-032 Gọi API tích hợp · UC-033 Xem phân tích hiệu năng tác tử



Hình 3.1. *Sơ đồ use case tổng thể của nền tảng*

Các quan hệ giữa use case phản ánh cấu trúc của quy trình: UC-004 *Thực thi quy trình* «include» UC-005 *Duyệt bước* khi quy trình có nút chờ con người; UC-006 *Từ chối* «extend» UC-005; UC-012 *Phát triển* «include» UC-023 *Phân tích chất lượng mã* (dự đoán phán quyết QA và sửa một vòng); UC-016 của QC dùng kết quả của UC-009 (use case do BA viết) làm đầu vào.

3.5 Đặc tả use case tiêu biểu

3.5.1 UC-002 Tạo dự án

- **Tác nhân:** thành viên hội đồng quản trị.
- **Tiền điều kiện:** đã đăng nhập, có quyền `project:create`.
- **Luồng chính:** (1) người dùng chọn mẫu quy trình và điền tham số (ví dụ số use case tối thiểu, có bật bước duyệt của con người không); (2) hệ thống kiểm tra mẫu và tham số trước khi tạo bất cứ thứ gì; (3) thực thể `Project` được tạo qua phương thức nhà máy `Project.create`, áp dụng các bất biến (tên không rỗng, ngân sách dương); (4) dự án được lưu, sự kiện `ProjectCreatedEvent` được phát; (5) bộ máy quy trình tạo một lượt chạy gắn với phiên bản bất biến `mẫu@phiên_bản+băm` của định nghĩa quy trình; (6) một dự báo thời gian và chi phí được lưu khi quy trình bắt đầu.
- **Luồng thay thế:** nếu mẫu không tồn tại, hệ thống trả về mã 404; nếu tham số sai kiểu, hệ thống trả về mã 400 trước khi tạo dự án; nếu người xem gọi chức năng này, hệ thống trả về mã 403 và ghi một mục `access.denied` vào nhật ký kiểm toán.
- **Hậu điều kiện:** dự án ở trạng thái `active`, có một lượt chạy quy trình; nhật ký kiểm toán có `project.create` và `workflow.started`.

3.5.2 UC-004 Thực thi quy trình

- **Tác nhân:** hệ thống (bộ máy quy trình) và các tác tử.
- **Luồng chính:** bộ máy lấy nút hiện tại, chọn chiến lược thực thi theo loại nút (`agent_task`, `sub_workflow`, `quality_gate`, `condition`, `parallel`, `human_approval`), ghi lại một bước (`WorkflowStep`) kèm thời lượng, gộp kết quả vào ngữ cảnh của lượt chạy, chụp ảnh trạng thái (Memento), chọn cạnh kế tiếp và lặp lại cho đến khi gặp nút kết thúc.
- **Luồng thay thế:** nút thất bại được thử lại tối đa ba lần với thời gian giãn cách tăng theo hàm mũ; nếu vẫn thất bại, lượt chạy đi theo cạnh `onFail` hoặc chuyển sang trạng thái `failed` và phát sự kiện `workflow.failed`; khi gặp nút chờ duyệt, lượt chạy chuyển sang trạng thái `paused`; khi chi phí vượt ngưỡng ngân sách, lượt chạy bị tạm dừng.

3.5.3 UC-016 Kiểm chứng hiện thực theo use case (QC)

- **Tác nhân:** tác tử QC.
- **Luồng chính:** (1) QC đọc các use case do BA viết và mã nguồn do DEV viết; (2) QC viết một tệp kiểm thử Jest cho mỗi use case (`tests/UC-xxx.spec.ts`); (3) hệ thống chạy kiểm thử trong sandbox Docker, hoặc trong một bản sao tạm thời nếu được phép chạy cục bộ; (4) QC xây dựng ma trận truy vết từ use case đến lớp, tệp hiện thực, tệp kiểm thử và kết quả kiểm thử; (5) độ bao phủ use case được ghi vào siêu dữ liệu OOAD của dự án.
- **Hậu điều kiện:** độ bao phủ chỉ tính các kiểm thử đã chạy và đạt; kiểm thử mới được viết mà chưa chạy đạt thì không được tính. Use case chưa có kiểm thử được liệt kê là còn thiếu.

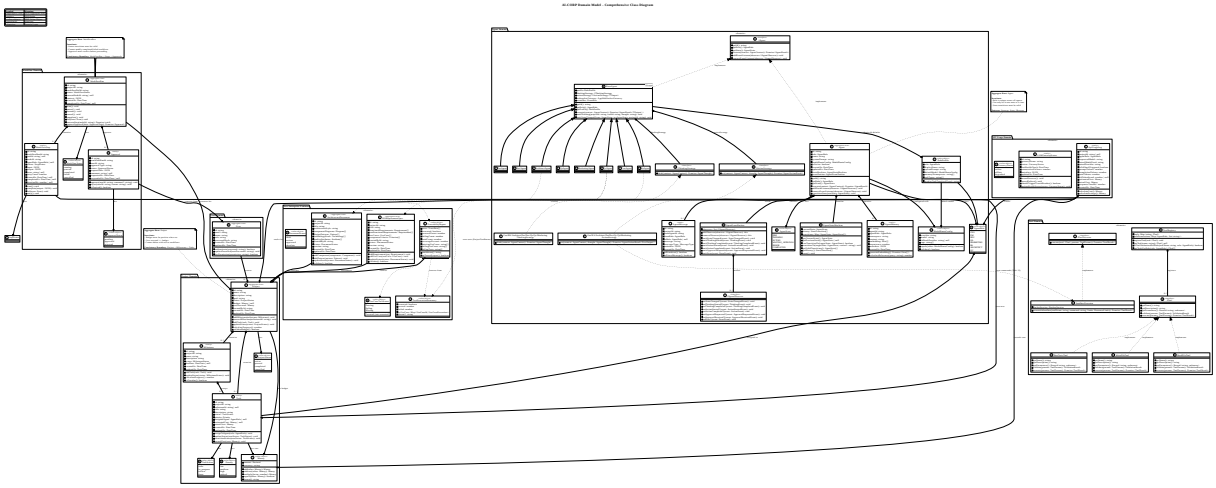
3.6 Mô hình miền

Mô hình miền của nền tảng gồm ba aggregate chính và một số khái niệm hỗ trợ:

- **Project** (gốc aggregate) chứa các `Milestone` và `Task`; có ngân sách và chi phí tích luỹ được biểu diễn bằng value object `Money`; trạng thái `ProjectStatus` là một value object có máy trạng thái riêng.
- **Agent** có `AgentRole`, cấu hình mô hình, bộ nhớ (`AgentMemory`) và máy trạng thái vòng đời.
- **WorkflowRun** tham chiếu đến một phiên bản bất biến của định nghĩa quy trình, chứa các `WorkflowStep` và các ảnh chụp `WorkflowSnapshot`, có thể có lượt chạy

cha (trường hợp quy trình con) và các `Approval`.

Các khái niệm hỗ trợ gồm `User` và `Role`, `AuditLog` (chỉ ghi thêm), `ApiKey`, `Webhook` và `WebhookDelivery`, `ApiUsageLog` (chi phí LLM), `QualityGateLog`, `ProjectEstimate`.



Hình 3.2. *Mô hình miền tổng quan*

Chương 4
THIẾT KẾ HỆ THỐNG

Chương này trình bày thiết kế của nền tảng theo trình tự từ tổng thể đến chi tiết, gồm kiến trúc, mẫu thiết kế, thiết kế tác tử, bộ máy quy trình, dữ liệu, cách áp dụng OOAD cho các dự án sinh ra, bảo mật, phân tích và dự báo, và cuối cùng là triển khai. Danh sách đầy đủ 39 sơ đồ UML được trình bày ở Phụ lục A.

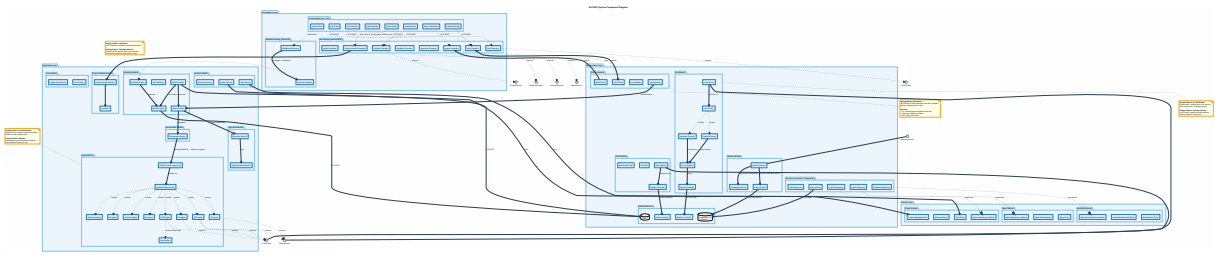
4.1 Kiến trúc tổng thể

4.1.1 Kiểu kiến trúc

Nền tảng gồm một máy chủ API (NestJS, TypeScript) và một ứng dụng web một trang (React). Bên trong API, mã nguồn được tổ chức theo kiến trúc phân lớp kết hợp với thiết kế hướng miền [6], [8], như trình bày trong bảng sau.

Tầng	**Nội dung**	**Phụ thuộc được phép**
Trình bày	Controller REST, cổng WebSocket (Socket.IO), DTO, guard	Ứng dụng
Ứng dụng	Dịch vụ ứng dụng (một use case một phương thức), chiến lược tác tử, bộ máy quy trình, trình xử lý sự kiện	Miền
Miền (`src/domain`)	Thực thể, value object, aggregate, sự kiện miền, máy trạng thái, giao diện Repository và các cổng (port), chính sách thuần (không I/O)	Không gì cả
Hạ tầng	Repository Prisma, nhà cung cấp LLM, sandbox Docker, kho tệp của dự án, phân tích mã TypeScript, xuất PDF	Miền (hiện thực các cổng)

Tầng miền gồm 106 tệp và không nhập NestJS, Prisma hay bất kỳ thư viện I/O nào. Nhờ đó, mọi quy tắc nghiệp vụ (chuyển trạng thái công việc, tính toán tiền tệ, phát hiện bất thường, hồi quy dự báo, chính sách chất lượng mã) đều được kiểm thử như các hàm thuần. Tầng miền cũng là nơi áp dụng ngưỡng độ bao phủ cao nhất (mục 6.2).



Hình 4.1. *Sơ đồ thành phần*

4.1.2 Các quyết định kiến trúc chính

Các quyết định kiến trúc được ghi thành các bản ghi quyết định kiến trúc (ADR) trong tệp `docs/design/architecture.md`. Sáu quyết định có ảnh hưởng lớn nhất là:

1. Dùng NestJS cho API, vì cơ chế tiêm phụ thuộc và hệ thống mô-đun của NestJS hiện thực trực tiếp nguyên lý DIP và cho phép đăng ký nhiều hiện thực cho cùng một token (được dùng cho danh sách tác tử và danh sách chiến lược nút).
2. Dùng PostgreSQL cho dữ liệu quan hệ và dữ liệu JSON (ngữ cảnh lượt chạy, siêu dữ liệu OOAD), với Prisma làm lớp ánh xạ phía sau các Repository.
3. Dùng BullMQ trên Redis làm hàng đợi cho các nút quy trình, kèm chế độ dự phòng thực thi trực tiếp khi không có Redis; khi đó điểm kiểm tra sức khoẻ báo trạng thái *degraded* và hệ thống vẫn tiếp tục hoạt động.
4. Mọi truy cập dữ liệu từ tầng ứng dụng đều đi qua Repository, nhờ đó các dịch vụ ứng dụng kiểm thử được bằng Repository trong bộ nhớ.
5. Dùng mẫu Strategy để chuyên môn hoá tác tử, thay cho cây kế thừa sâu (mục 4.3).
6. Các mô-đun giao tiếp với nhau qua sự kiện miền. Nhật ký kiểm toán, webhook, cơ chế học của tác tử, dự báo và giám sát bất thường đều là bộ lắng nghe sự kiện, và các mô-đun này không gọi trực tiếp lẫn nhau.

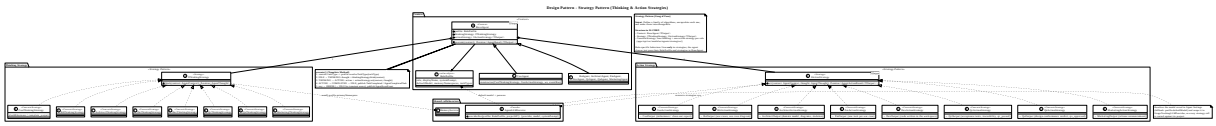
4.2 Mẫu thiết kế

Bảng dưới đây gắn mỗi mẫu thiết kế với vấn đề mà mẫu đó giải quyết và lớp hiện thực tương ứng. Mỗi lớp trong mã nguồn đều có chú thích `@pattern`, vì vậy có thể kiểm chứng nội dung bảng bằng cách tìm kiếm trong kho mã.

Vấn đề	**Mẫu**	**Hiện thực**
Tám vai trò có chung chu trình nhận việc, suy nghĩ, hành động và hoàn tất	Template Method	`BaseAgent.execute()`

Vấn đề	Mẫu	Hiện thực
Mỗi vai trò suy nghĩ và hành động khác nhau, cần thay được độc lập	Strategy	`IThinkingStrategy`, `IActionStrategy`, 16 lớp chiến lược
Thêm vai trò mới không sửa bộ điều phối	Registry + DIP	`AgentOrchestrator` nhận danh sách `IAgent` qua token `AGENTS`; `RoleProfile` cho mỗi vai trò
Vòng đời tác tử, công việc, dự án, phê duyệt có chuyển trạng thái hợp lệ	State	`AgentStateMachine`, `Task.transitionTo`, value object `ProjectStatus`
Nhiều bên cần biết khi tác tử đổi trạng thái hoặc hoàn tất	Observer	`AgentEventEmitter` / `IAgentObserver`; `EventBus` với các handler
Mỗi loại nút quy trình thực thi khác nhau, thêm loại nút không sửa bộ máy	Strategy (tự đăng ký)	`NodeExecutionStrategy`: tác tử, quy trình con, cổng chất lượng, điều kiện, song song, phê duyệt
Một nút là cả một quy trình	Composite	`SubWorkflowStrategy` chạy quy trình con như lượt chạy con
Quay lại trạng thái trước của lượt chạy	Memento	`WorkflowRunMemento`, `WorkflowSnapshotCaretaker`
Định nghĩa quy trình không bị sửa sau khi đã có lượt chạy dùng nó	Immutable registry	`WorkflowDefinitionRegistry` theo `id@phiên_bản+băm`
Ghi chi phí mọi lời gọi LLM mà không sửa nhà cung cấp	Decorator	`UsageTrackingLLMProvider`
Tác tử gọi LLM qua một điểm duy nhất (cấu hình, nhà cung cấp, chi phí)	Facade	`AgentLLMService`, `OoadArtifactService`, `CodeQualityService`
Nhiều nhà cung cấp LLM sau một giao diện	Adapter + Factory Method	`ILLMProvider`, `LLMProviderFactory`
Quản trị viên thêm nhà cung cấp và mô hình lúc chạy, mỗi loại nhà cung cấp tạo client khác nhau	Aggregate + Strategy + Registry	`LlmProviderConfig` (sở hữu `LlmModel`), `ProviderStrategy` cho mỗi loại, `ProviderStrategyRegistry`, `ConfiguredProviderRegistry`
Khoá API của nhà cung cấp được mã hoá mà miền không phụ thuộc thư viện mật mã	Port / Adapter	`SecretCipher` (miền) ← `AesGcmSecretCipher` (hạ tầng)
Một phê duyệt chỉ được trả lời một lần dù có hai yêu cầu đồng thời	Cập nhật có điều kiện (compare-and-set)	`ApprovalService`: cập nhật chỉ khi trạng thái còn `pending`
Tác tử trao đổi thông điệp có kiểu khi cộng tác	Mediator	`AgentMessageBus`, `EventBus`

Vấn đề	Mẫu	Hiện thực
Xác thực, giới hạn tần suất rồi kiểm tra quyền	Chain of Responsibility	`AuthenticationGuard` → `RateLimitGuard` → `PermissionGuard`; chuỗi rà soát QC → QA
Sự kiện miền thành mục kiểm toán hoặc payload webhook công khai	Adapter	`AuditLogHandler`, `webhook-payloads`
Truy cập dữ liệu tách khỏi miền	Repository, Data Mapper	`I*Repository` trong miền, `Prisma*Repository` trong hạ tầng
Báo cáo chi phí và hiệu năng dựng từng bước	Builder	`CostReportBuilder`, `PerformanceReportBuilder`
Xuất JSON / CSV / PDF chọn theo định dạng	Strategy + Registry	`ProjectExportService` và ba exporter
Mỗi loại bất thường, mỗi tín hiệu chất lượng là một quy tắc	Strategy + Composite	`IAgentAnomalyRule`, `IQualitySignal`, `AgentAnomalyDetector`
Phân tích mã sinh ra mà miền không phụ thuộc trình biên dịch	Port / Adapter	`ICodeAnalyzer` (miền) ← `TypeScriptCodeAnalyzer` (hạ tầng)
Quy tắc chấm điểm chất lượng mã	Policy / Specification	`CodeQualityPolicy`, `LayerRules`



Hình 4.2. *Mẫu Strategy cho tác tử*

4.3 Thiết kế tác tử

4.3.1 Template Method và hai chiến lược

Mỗi tác tử là một đối tượng `BaseAgent` được cấu hình bằng ba thành phần: một `RoleProfile` (vai trò, các loại công việc được hỗ trợ, loại công việc mặc định, lời nhắc hệ thống), một chiến lược tư duy và một chiến lược hành động. Phương thức `BaseAgent.execute()` cố định khung thuật toán gồm các bước sau:

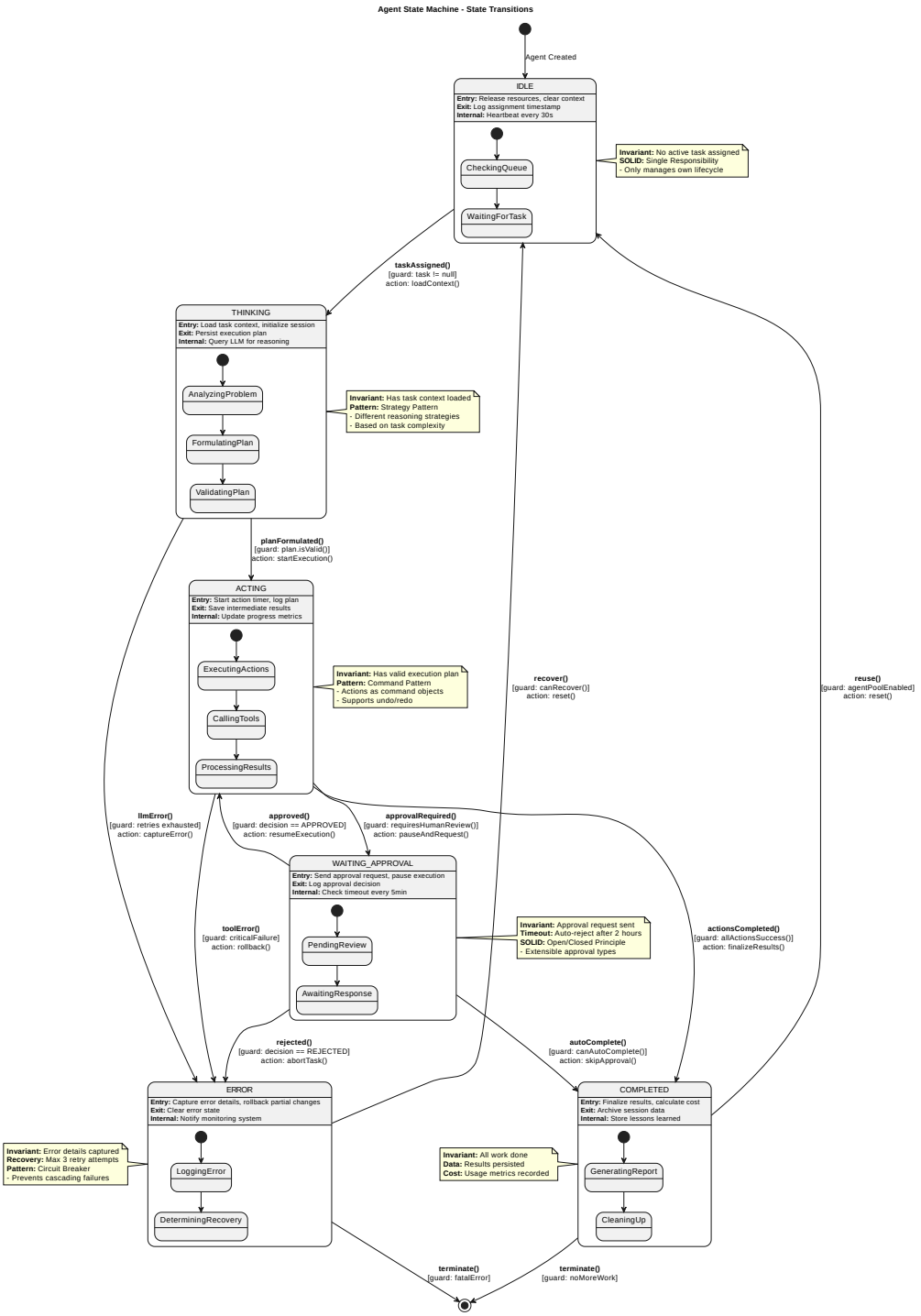
1. Chuẩn hoá ngữ cảnh: loại công việc được đối chiếu với hồ sơ vai trò, nhờ đó chiến lược luôn nhận được một loại công việc mà vai trò hỗ trợ.
2. Phát `TaskAssignedEvent`.
3. Chuyển trạng thái `IDLE → THINKING` và gọi `thinkingStrategy.think(context)`: nạp bộ nhớ và các bài học liên quan, sau đó gọi LLM để lập kế hoạch.

4. Chuyển trạng thái `THINKING → ACTING` và gọi `actionStrategy.act(context, thought)`: sinh tài liệu hoặc mã nguồn, ghi vào kho tệp của dự án và chạy các công cụ cần thiết.
5. Chuyển trạng thái `ACTING → COMPLETED → IDLE`, lưu ký ức và phát `AgentCompletedTaskEvent`.
6. Khi có ngoại lệ, tác tử chuyển sang trạng thái `ERROR`; với lỗi tạm thời, tác tử được phép quay lại `IDLE`.

Thiết kế này thoả mãn SRP (tác tử chỉ điều phối vòng đời, còn chuyên môn nằm trong các chiến lược), OCP (thêm một vai trò chỉ cần thêm một hồ sơ và hai chiến lược) và LSP (mọi tác tử có thể thay thế cho nhau thông qua giao diện `IAgent`).

4.3.2 Máy trạng thái theo từng lần thực thi

Trong thiết kế ban đầu, mỗi vai trò chỉ giữ một máy trạng thái duy nhất. Kiểm thử đã phát hiện hai hậu quả của cách làm này. Thứ nhất, hai công việc cùng vai trò chạy đồng thời làm hỏng trạng thái của nhau (lỗi "Invalid state transition: IDLE → ERROR"). Thứ hai, sau một lỗi vĩnh viễn, vai trò bị kẹt ở trạng thái `ERROR` và từ chối mọi công việc sau đó. Trong thiết kế đã sửa, mỗi lần gọi `execute()` có một máy trạng thái riêng; tác tử giữ tập các máy trạng thái đang chạy và báo cáo trạng thái của lần thực thi gần nhất. Trường hợp này cho thấy mẫu State cần được gắn với đối tượng thực sự có vòng đời, ở đây là lần thực thi, thay vì với đối tượng tồn tại lâu dài là tác tử.



Hình 4.3. *Máy trạng thái của tác tử*

4.3.3 Gọi LLM và ghi chi phí

Các chiến lược không gọi trực tiếp nhà cung cấp LLM. `AgentLLMService` (mẫu Facade) tra cứu cấu hình mô hình của vai trò, lấy nhà cung cấp từ `LLMProviderFactory` và bọc nhà cung cấp đó bằng `UsageTrackingLLMProvider` (mẫu Decorator). Decorator chuyển tiếp nguyên vẹn lời gọi, sau đó ghi số token và chi phí vào `ApiUsageLog`, cộng vào chi phí của dự án và phát sự kiện `BudgetExceededEvent` một lần khi vượt ngân

sách. Nhờ vậy, chi phí của cả tám vai trò được ghi nhận đầy đủ mà không chiến lược nào phải xử lý việc tính chi phí.

Nhà cung cấp và mô hình LLM được quản lý ngay trong ứng dụng, ngoài cách cấu hình qua biến môi trường. `LlmProviderConfig` là một aggregate root: đối tượng này sở hữu danh sách `LlmModel`, kiểm tra các bất biến (tên dài từ 1 đến 60 ký tự, URL http hoặc https hợp lệ, loại nhà cung cấp đã được hỗ trợ, không có mô hình trùng lặp) và chỉ lưu khoá API ở dạng đã mã hoá. Việc mã hoá đi qua cổng `SecretCipher` của tầng miền; tầng hạ tầng hiện thực cổng này bằng thuật toán AES-256-GCM với khoá `LLM_SECRET_KEY`, và API chỉ trả về bốn ký tự cuối của khoá. Mỗi loại nhà cung cấp (tương thích OpenAI, Anthropic) là một `ProviderStrategy` có khả năng tạo client, kiểm tra kết nối và lấy danh sách mô hình; để hỗ trợ một loại mới, chỉ cần thêm một chiến lược vào sổ đăng ký. Cấu hình mô hình của mỗi vai trò có thêm trường `providerId`, và `LLMProviderFactory` truy vấn `ConfiguredProviderRegistry` để lấy client đã cấu hình (có bộ đệm), hoặc dùng cổng mặc định khi vai trò chưa chọn nhà cung cấp. Ở lần khởi động đầu tiên, nếu bảng nhà cung cấp còn trống, hệ thống nhập cổng LLM đang cấu hình trong biến môi trường thành nhà cung cấp đầu tiên, nhờ đó một bản cài đặt cũ không phải cấu hình lại. Tác tử, các chiến lược và decorator ghi chi phí không phải thay đổi khi bổ sung tính năng này, vì chúng chỉ phụ thuộc vào giao diện `ILLMProvider`.

4.3.4 Học từ kết quả

Cơ chế học gồm ba thành phần:

1. `OutcomeSignalPolicy` (một chính sách thuần) xác định tín hiệu nào được dùng để đánh giá lần thực thi nào: kết quả của cổng OOAD đánh giá BA và Architect; kết quả của QC (số kiểm thử đạt, các use case còn thiếu) đánh giá DEV; phán quyết của QA đánh giá DEV và QC.
2. `LearningService` lưu mỗi kết quả vào bộ nhớ của vai trò; cứ sau N kết quả (mặc định là 3), dịch vụ này chắt lọc các kết quả gần đây thành một ký ức loại `lessons_learned` bằng một lời gọi LLM, và lời gọi này cũng được tính chi phí như mọi lời gọi khác.
3. Khi xây dựng lời nhắc, chiến lược tư duy nạp các bài học này trước tiên.

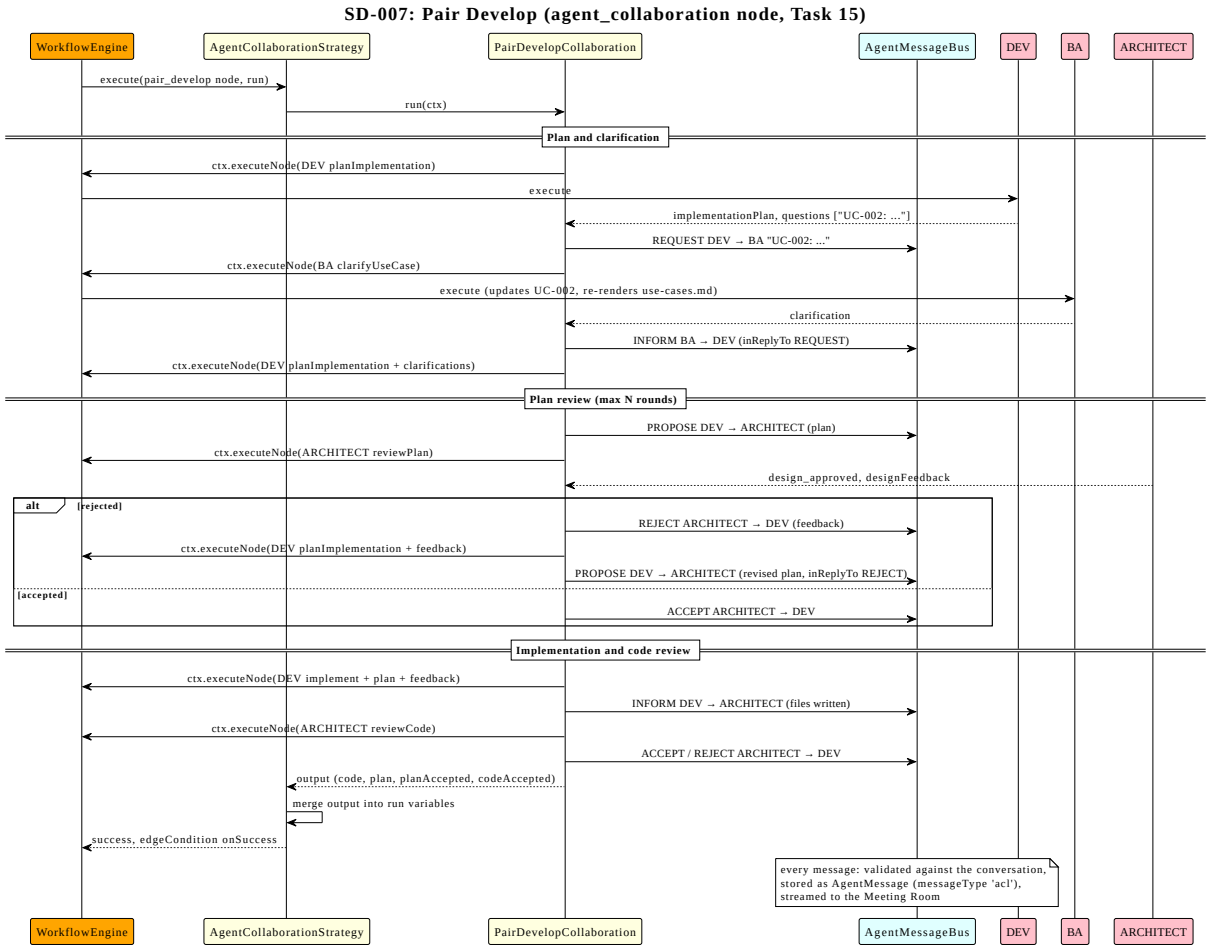
Thiết kế này gần với Reflexion [28] ở chỗ phản hồi được biểu diễn bằng ngôn ngữ tự nhiên. Điểm khác là tín hiệu đánh giá đến từ nguồn khách quan (cổng chất lượng, kết quả kiểm thử, phán quyết QA), trong khi ở Reflexion tác tử tự đánh giá chính mình.

4.3.5 Cộng tác giữa các tác tử

Ba mẫu cộng tác được hiện thực dưới dạng các chiến lược của một nút quy trình và được đăng ký trong `CollaborationRegistry`:

- **Lập trình cặp** (`pair_develop`): DEV lập kế hoạch; các câu hỏi về use case còn mơ hồ được gửi cho BA trước (vòng làm rõ); DEV gửi thông điệp `PROPOSE`, Architect trả lời `ACCEPT` hoặc `REJECT` kèm phản hồi; sau khi DEV hiện thực, Architect rà soát mã. Số vòng trao đổi được giới hạn để chi phí không vượt quá mức cho phép.
- **Vòng làm rõ** (`clarification_loop`): tác tử hỏi BA và nhận câu trả lời trước khi tiếp tục.
- **Chuỗi rà soát** (`review_chain`, mẫu Chain of Responsibility): QC rà soát trước, sau đó đến QA; nếu một mắt xích từ chối thì chuỗi dừng lại.

Mọi lượt trao đổi đều đi qua `AgentMessageBus` (mẫu Mediator) với các performative theo tinh thần của FIPA-ACL [16], và được hiển thị trong màn hình Phòng họp (Meeting Room) của giao diện.



Hình 4.4. *Cộng tác lập trình cặp có vòng làm rõ*

4.4 Thiết kế bộ máy quy trình

4.4.1 Định nghĩa và thực thi

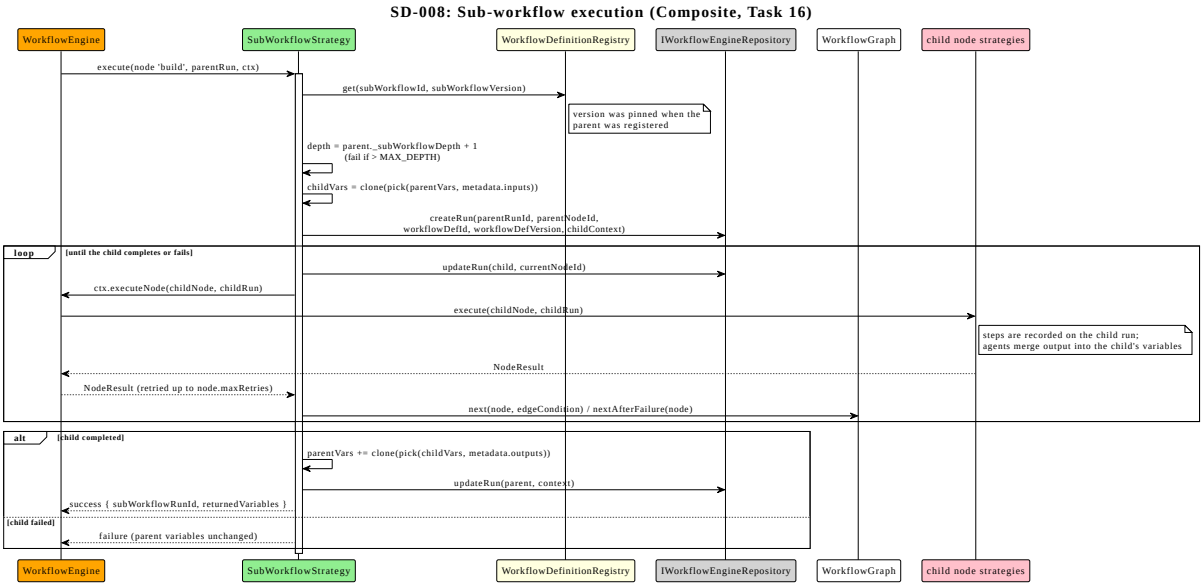
Một quy trình là một đồ thị gồm các nút có kiểu và các cạnh (cạnh mặc định, cạnh có điều kiện, cạnh `onFail`). `WorkflowEngine` không chứa logic riêng của từng loại nút; mỗi loại nút là một `NodeExecutionStrategy` tự đăng ký với bộ máy. Do đó, để thêm một loại nút mới (ví dụ cổng chất lượng), chỉ cần thêm một lớp mà không phải sửa bộ máy, đúng với nguyên lý OCP.

Quy trình mặc định cho một dự án phần mềm như sau:

```
CEO (mốc)
 → [quy trình con: phân tích & thiết kê]  BA → Architect → cổng OOAD —không đạt→
Architect
 → PM (một công việc cho môĩ use case)
 → [quy trình con: xây dựng & kiểm chứng] DEV → QC → QA → [điều kiện qa_approved]
—không đạt→ DEV
 → Marketing → [phê duyệt của con người] → CEO (đóng dự án)
```

4.4.2 Quy trình con, phiên bản và khôi phục

- **Quy trình con (Composite):** nút `sub_workflow` chạy một quy trình khác dưới dạng một lượt chạy con có `parentRunId`; kết quả của lượt chạy con được gộp vào ngữ cảnh của lượt chạy cha. Ba mẫu quy trình có kiểu (dự án phần mềm OOAD, backend REST API, chỉ phân tích và thiết kế) dùng chung các quy trình con này.
- **Phiên bản bất biến:** mỗi định nghĩa quy trình được lưu theo khoá `id@phiên_bản` cùng với mã băm của nội dung đã chuẩn hoá; việc đăng ký lại cùng một phiên bản với nội dung khác sẽ bị từ chối. Mỗi lượt chạy được gắn cố định với một phiên bản cụ thể.
- **Ảnh chụp và khôi phục (Memento):** sau mỗi nút, `WorkflowSnapshotCaretaker` lưu một memento gồm nút hiện tại, ngữ cảnh và trạng thái. Người dùng có thể khôi phục một lượt chạy đang tạm dừng về một ảnh chụp trước đó; khi đó các yêu cầu phê duyệt đang treo phát sinh sau ảnh chụp sẽ bị huỷ.
- **Song song có phụ thuộc:** nút `parallel` chạy các nhánh theo thứ tự tô-pô xác định bởi `dependsOn`, với giới hạn số nhánh chạy đồng thời; `DependencyResolver` phát hiện các chu trình phụ thuộc.



Hình 4.5. *Thực thi quy trình con*

4.4.3 Xử lý lỗi

Một nút thất bại được thử lại tối đa ba lần với thời gian giãn cách tăng theo hàm mũ. Nếu vẫn thất bại, lượt chạy đi theo cạnh `onFail` nếu có; nếu không, lượt chạy chuyển sang trạng thái `failed` và phát sự kiện `workflow.failed`. Kiểm thử đã phát hiện và

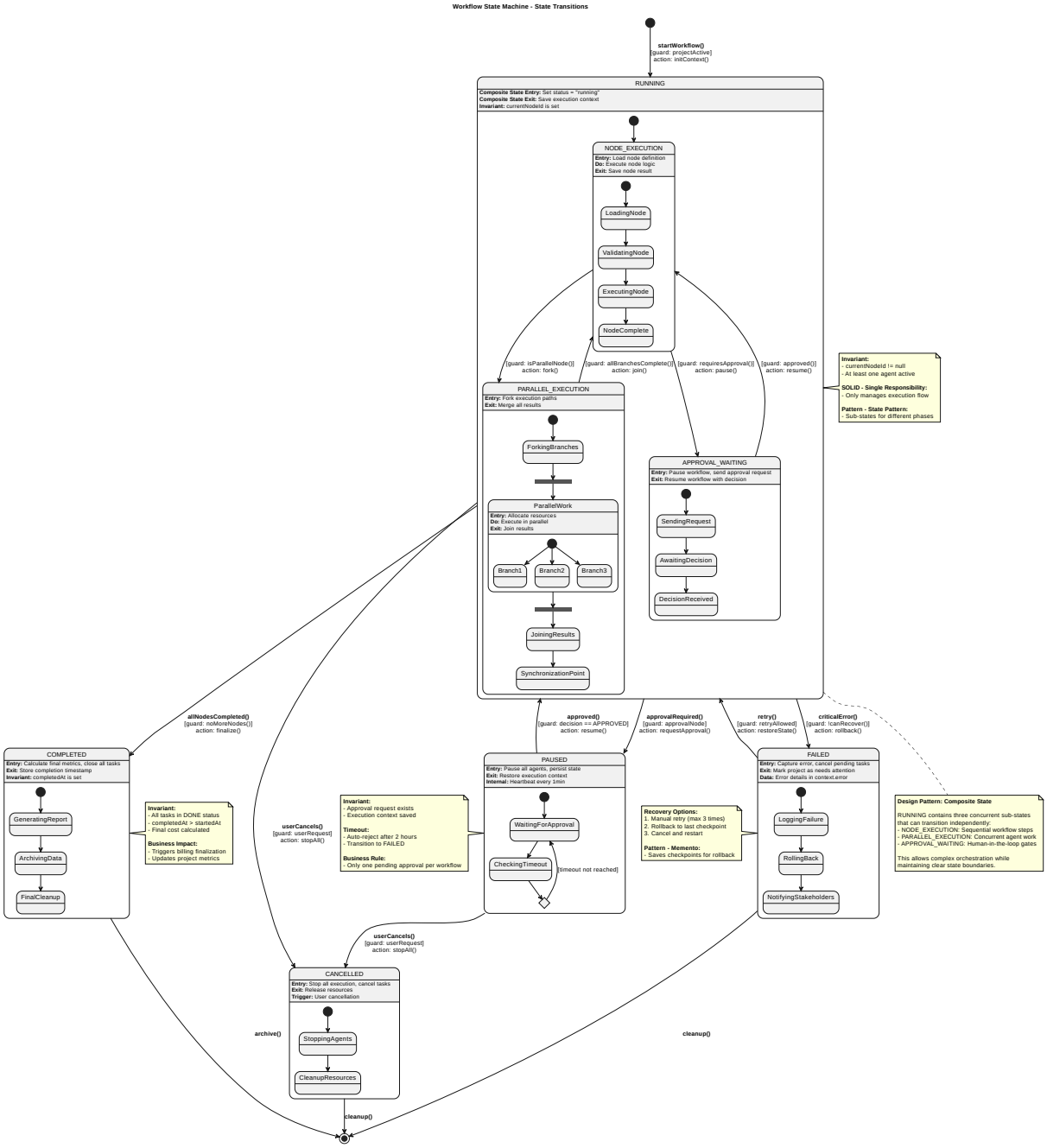
giúp sửa hai lỗi thiết kế liên quan: trước đây, một nút thất bại có thể khiến lượt chạy BullMQ ở mãi trạng thái `running`, hoặc khiến lượt chạy đi tiếp theo cạnh thành công.

4.4.4 Điều khiển lượt chạy và ngân sách

Các thao tác tạm dừng, tiếp tục và huỷ tác động lên lượt chạy mới nhất của dự án, ngoài việc cập nhật trạng thái của dự án; khi xoá dự án, hệ thống huỷ lượt chạy và xoá kho tệp của dự án đó. Ba quy tắc sau bảo đảm các thao tác này hoạt động đúng:

- Không cho phép tiếp tục khi lượt chạy đang chờ phê duyệt (vì khi đó nút phê duyệt sẽ chạy lại và yêu cầu phê duyệt hai lần), cũng như khi ngân sách đã hết; trong cả hai trường hợp, hệ thống trả về lỗi kèm lý do.
- Không nút nào được bắt đầu khi ngân sách đã hết. Bộ máy kiểm tra ngân sách trước mỗi nút và tạm dừng lượt chạy nếu ngân sách đã hết. Chi phí được cộng dồn bằng phép tăng nguyên tử trong cơ sở dữ liệu thay cho cách đọc rồi ghi, và các thông báo ở mức 90% và 100% chỉ được phát một lần, tại thời điểm vượt ngưỡng.
- Một nút đã kết thúc trong lúc lượt chạy đang tạm dừng sẽ không bị chạy lại khi lượt chạy tiếp tục.

Các quy tắc này được bổ sung sau một đợt rà soát mã có hệ thống đối với phần giao diện và logic (mục 6.9). Trước đó, nút *Pause* trên giao diện chỉ thay đổi trạng thái của dự án trong khi lượt chạy vẫn tiếp tục, và ngân sách chỉ được kiểm tra sau khi chi phí đã phát sinh.



Hình 4.6. *Máy trạng thái của quy trình*

4.5 Thiết kế dữ liệu và lưu trữ

Dữ liệu có cấu trúc được lưu trong PostgreSQL thông qua 22 mô hình Prisma (người dùng, dự án, mốc, công việc, tác tử, bộ nhớ, lượt chạy, bước, ảnh chụp, phiên bản định nghĩa, phê duyệt, nhật ký sử dụng API, nhật ký cổng chất lượng, nhật ký kiểm toán, khoá API, webhook, lần gửi webhook, dự báo, nhà cung cấp LLM, mô hình LLM, v.v.). Lược đồ cơ sở dữ liệu được quản lý bằng các migration có phiên bản; một bước kiểm tra so sánh lược đồ dựng từ migration với mô hình để phát hiện sai lệch.

Sản phẩm của tác tử (tài liệu, sơ đồ, mã nguồn, kiểm thử, báo cáo) được lưu trong `ProjectWorkspace` thay vì trong cơ sở dữ liệu. Đây là một thư mục lưu trữ lâu dài cho mỗi dự án, có cơ chế chống thoát khỏi thư mục (ví dụ, đường dẫn `../../etc/...` do LLM đề xuất sẽ bị từ chối). Thiết kế ban đầu ghi sản phẩm vào sandbox Docker; sandbox này chạy mỗi lệnh trong một container mới không gắn ổ lưu trữ, nên mọi tệp đều bị mất sau mỗi lệnh. Đó là lý do cổng chất lượng và bước quét kiểm thử trong thiết kế ban đầu chưa bao giờ chạy thành công.

4.6 Áp dụng OOAD cho dự án sinh ra

Mọi dự án do nền tảng sinh ra đều đi qua cùng một chuỗi OOAD, và từng bước trong chuỗi đều được kiểm tra tự động.

4.6.1 Chuỗi sản phẩm

Bước	**Tác tử**	**Sản phẩm trong kho tệp của dự án**
Yêu cầu	BA	`docs/use-cases.md`
Mô hình miền và UML	Architect	`docs/domain-model.puml`, `docs/use-case-diagram.puml`, `docs/class-diagram.puml`, `docs/sequence-UC-xxx.puml`, `docs/architecture-patterns.md`, `ARCHITECTURE.md`
Khung mã	Architect	`src/domain/{entities,value-objects,repositories}`, `src/application/services`, `src/infrastructure/persistence`, `src/presentation/controllers`, `src/composition-root.ts`
Hiện thực	DEV	Mã đầy đủ trong khung, chú thích `UC-xxx`
Chất lượng mã	Hệ thống	`docs/code-quality.md`
Kiểm thử	QC	`tests/UC-xxx.spec.ts`, kết quả Jest
Truy vết	QC	`docs/traceability-matrix.md`

Khung mã được dựng từ các mẫu Handlebars theo đúng bốn tầng: giao diện Repository nằm trong tầng miền, phần hiện thực nằm trong tầng hạ tầng, và tệp `composition-root.ts` lắp ghép các phụ thuộc. Như vậy, kiến trúc mà nền tảng áp dụng cho chính mình cũng là kiến trúc nền tảng yêu cầu ở sản phẩm.

4.6.2 Ba lớp kiểm tra

1. **Cổng OOAD** (nút `quality_gate` sau bước thiết kế): kiểm tra số use case, số thực thể miền và số mẫu thiết kế có đạt ngưỡng hay không, và các tệp bắt buộc có tồn tại hay không. Nếu không đạt, quy trình quay lại bước của Architect. Mỗi lần

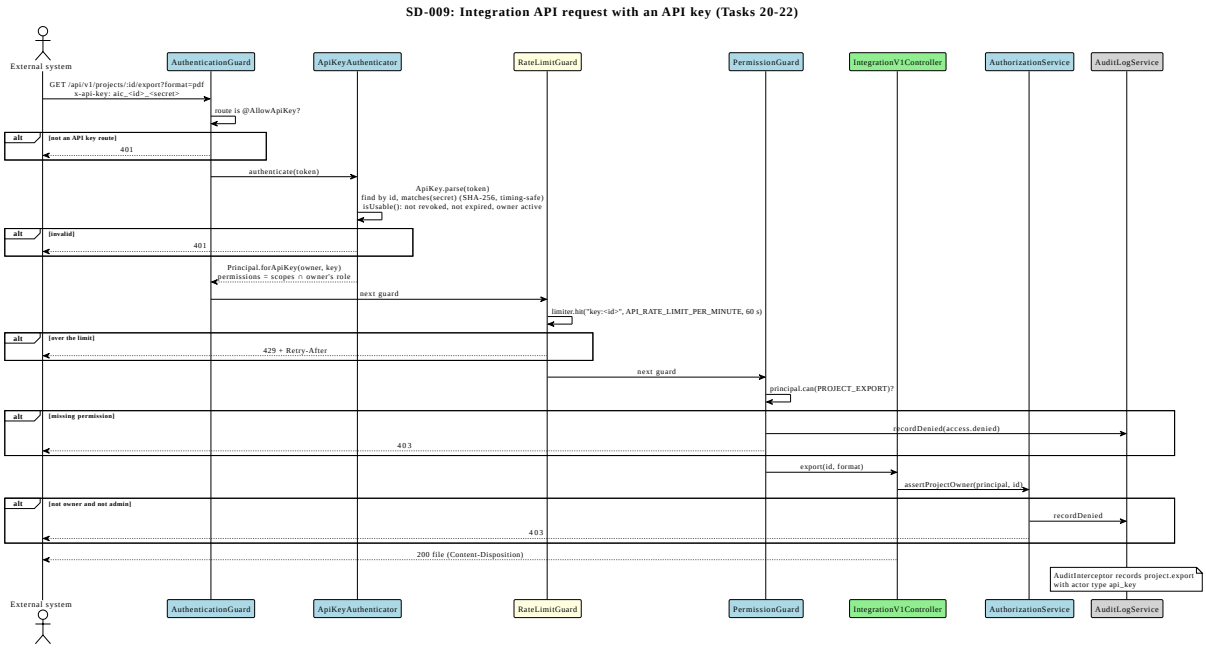
đánh giá được ghi vào `QualityGateLog` và phát sự kiện `QualityGateEvaluatedEvent`.

2. **Phân tích tĩnh trước khi QA rà soát** (mục 4.8.3), trong đó có luật phân lớp *miền ← ứng dụng ← hạ tầng/trình bày*: mọi lệnh `import` hoặc `require()` từ tầng miền vào tầng hạ tầng đều bị coi là vi phạm.
3. **Kiểm thử chấp nhận đã thực thi**: độ bao phủ use case chỉ tính các use case có kiểm thử đã chạy và đạt; các use case thiếu kiểm thử hoặc có kiểm thử thất bại được liệt kê cụ thể.

4.7 Bảo mật và tính năng doanh nghiệp

- **Chuỗi guard toàn cục** (mẫu Chain of Responsibility): `AuthenticationGuard` (JWT [49] cho người dùng, khoá API cho `/api/v1`) → `RateLimitGuard` → `PermissionGuard`. Theo mặc định, mọi route đều yêu cầu đăng nhập; route công khai phải được đánh dấu bằng `@Public`.
- **Quyền sở hữu**: `AuthorizationService` kiểm tra chủ sở hữu dự án và người được giao phê duyệt; khi vi phạm, hệ thống trả về mã 403 và ghi sự kiện `access.denied`. Việc tạo và sửa công việc cũng được kiểm tra quyền chủ sở hữu giống như việc sửa dự án.
- **Đặc quyền tối thiểu khi đăng ký**: tài khoản tự đăng ký chỉ nhận vai trò người xem (`SIGNUP_ROLE`); quản trị viên nâng quyền trên trang quản lý người dùng và có thể tắt chức năng đăng ký (`ALLOW_REGISTRATION=false`). Phiên đăng nhập hết hạn sau khoảng thời gian `JWT_EXPIRATION`, mặc định là 8 giờ.
- **Nhật ký kiểm toán chỉ ghi thêm**: decorator `@Audited` cùng với `AuditInterceptor` ghi lại hành động của người dùng; `AuditLogHandler` ghi lại hành động của hệ thống và tác tử dựa trên sự kiện miền. Mật khẩu, token, bí mật và khoá được che trước khi ghi.
- **API tích hợp**: khoá API chỉ mang các phạm vi mà vai trò hiện tại của chủ khoá cho phép (giao của hai tập phạm vi); tần suất gọi được giới hạn theo từng khoá; tài liệu OpenAPI được cung cấp tại `/api/docs` (có thể tắt trong môi trường production).
- **Webhook**: payload được ký bằng HMAC-SHA256 [48] kèm dấu thời gian; URL trỏ tới mạng nội bộ bị từ chối, kể cả sau khi phân giải DNS (chống tấn công SSRF [50]); việc gửi lại được thực hiện với thời gian giãn cách tăng theo hàm mũ.

- **Phê duyệt chỉ qua REST**: kiểm thử đầu-cuối trên cấu hình triển khai phát hiện rằng giao diện gửi quyết định phê duyệt qua WebSocket, và cổng WebSocket chấp nhận quyết định từ bất kỳ kết nối nào. Kết quả là người xem, vốn bị REST trả về mã 403, vẫn phê duyệt được, và dự án được đóng mà không có mục kiểm toán nào. Trong thiết kế đã sửa, quyết định phê duyệt chỉ đi qua `POST /api/approvals/:id/approve|reject` (có kiểm tra quyền, kiểm tra người được giao và ghi kiểm toán), còn thông điệp WebSocket tương ứng bị từ chối. Việc trả lời phê duyệt là một cập nhật có điều kiện trên trạng thái `pending`, nên hai câu trả lời đồng thời không thể cùng thành công (câu trả lời thứ hai nhận mã 409). Quyết định và lý do (bắt buộc khi từ chối) được ghi vào biến của lượt chạy để các tác tử chạy sau có thể đọc.
- **Cấu hình production an toàn**: khi `NODE_ENV=production`, API sẽ không khởi động nếu thiếu `JWT_SECRET` (hoặc giá trị này ngắn hơn 32 ký tự), thiếu `CORS_ORIGIN`, hoặc vẫn còn giá trị giữ chỗ như `CHANGE_ME`. Mọi nơi đọc khoá JWT (REST, guard WebSocket, cổng WebSocket) đều dùng chung một hàm. Trước đây, ba nơi này có ba giá trị dự phòng khác nhau, nên khi không đặt `JWT_SECRET`, kết nối thời gian thực luôn bị từ chối.



Hình 4.7. *Chuỗi xử lý một yêu cầu có khoá API*

4.8 Phân tích, dự báo và chất lượng mã

4.8.1 Phân tích hiệu năng tác tử

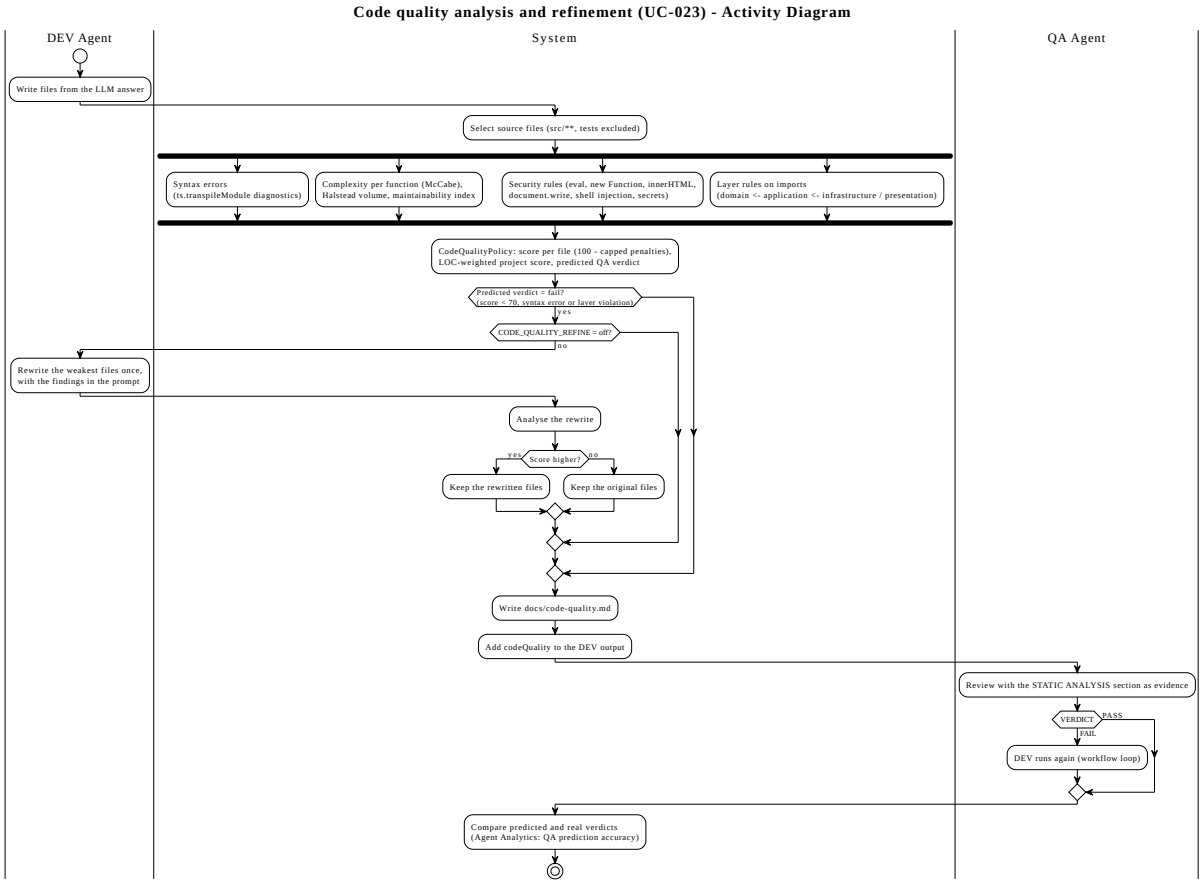
`AgentPerformanceAnalyzer` (thuộc tầng miền, là hàm thuần) tính các chỉ số sau cho mỗi vai trò: số bước, tỉ lệ thành công, thời lượng trung bình và phân vị 95, chi phí cho mỗi lần thành công, chất lượng, xu hướng so với kỳ trước và thứ hạng. Chất lượng được đo theo từng vai trò bằng các `IQualitySignal` khác nhau: DEV được đo theo tỉ lệ được QA duyệt, QC theo kết quả kiểm thử, Architect theo kết quả cổng OOAD, còn các vai trò khác theo tỉ lệ thành công ngay từ lần đầu. `AgentAnomalyDetector` áp dụng năm quy tắc: tỉ lệ thành công giảm 20 điểm phần trăm, ba lần thất bại liên tiếp, chi phí tăng gấp đôi, thời gian chạy tăng gấp đôi, và chất lượng dưới 50%. Mỗi bất thường được phát tối đa một lần mỗi ngày qua cùng kênh sự kiện, kiểm toán và webhook.

4.8.2 Dự báo thời gian và chi phí

Nền tảng không ghi nhận giờ công của con người mà ghi nhận thời lượng và chi phí LLM của từng bước do tác tử thực hiện. Vì vậy, mô hình dự báo thời gian của tác tử và chi phí LLM theo từng dự án và từng vai trò, thay cho giờ công của từng công việc. Mỗi vai trò có một mô hình hồi quy ridge [40] trên hai đặc trưng là độ dài mô tả dự án và số use case; khi một vai trò có ít hơn 6 mẫu, mô hình dùng giá trị trung vị. Độ chính xác được báo cáo bằng sai số leave-one-out (MAPE và MAE) [46], và mỗi dự báo đi kèm một mức tin cậy. Dự báo được lưu khi quy trình bắt đầu và khi PM lập kế hoạch, sau đó được so sánh với kết quả thực tế khi dự án hoàn thành. Chi phí ước lượng của mỗi công việc do PM tạo ra được suy ra từ dự báo của DEV, QC và QA.

4.8.3 Phân tích tĩnh và vòng sửa có giới hạn

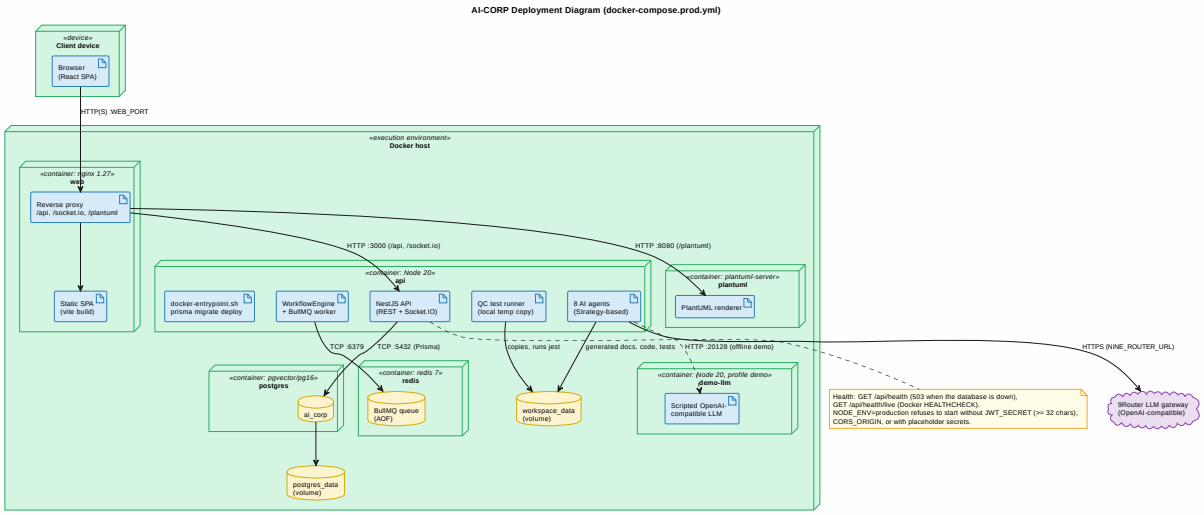
`TypeScriptCodeAnalyzer` (thuộc tầng hạ tầng, nằm sau cổng `ICodeAnalyzer`) dùng TypeScript compiler API để xác định: lỗi cú pháp, độ phức tạp McCabe của từng hàm [35], thể tích Halstead [36], chỉ số bảo trì đã chuẩn hoá [37], [38], các lời gọi không an toàn (`eval`, `Function`), bí mật được viết cứng trong mã, vi phạm phân lớp và các tham chiếu đến use case. `CodeQualityPolicy` (thuộc tầng miền) trừ điểm có giới hạn trần cho mỗi loại vấn đề, tính điểm trung bình có trọng số theo số dòng, và dự đoán phán quyết của QA là *fail* khi điểm dưới 70, khi có lỗi cú pháp hoặc khi có vi phạm phân lớp. Khi dự đoán là *fail*, DEV được thực hiện đúng một vòng sửa với danh sách vấn đề kèm theo; bản sửa chỉ được giữ lại nếu điểm tăng. QA nhận báo cáo phân tích làm bằng chứng khi rà soát. Dự đoán này được đối chiếu với phán quyết thực tế để đo độ chính xác.



Hình 4.8. *Luồng phân tích chất lượng mã*

4.9 Thiết kế triển khai

Hệ thống được triển khai bằng Docker Compose trên một máy chủ, gồm các dịch vụ: nginx phục vụ ứng dụng web và làm proxy ngược cho `/api`, `/socket.io` và `/plantuml`; API (Node 20) chạy migration khi khởi động rồi bắt đầu phục vụ; PostgreSQL (có pgvector); Redis (chế độ AOF); và máy chủ PlantUML nội bộ. Kho tệp của các dự án là một volume. Chỉ có nginx được công bố ra bên ngoài. Hồ sơ `demo` bổ sung LLM kịch bản để trình diễn khi không có kết nối mạng. API có hai điểm kiểm tra sức khoẻ [54]: `/api/health/live` cho biết tiến trình còn phản hồi và được dùng cho HEALTHCHECK của Docker; `/api/health` cho biết hệ thống đã sẵn sàng, trả về mã 503 khi mất kết nối cơ sở dữ liệu và trạng thái *degraded* khi mất Redis hoặc cổng LLM.



Hình 4.9. *Sơ đồ triển khai*

Chương 5
HIỆN THỰC

Chương này mô tả việc hiện thực thiết kế đã trình bày ở chương 4, gồm công nghệ sử dụng, quy mô mã nguồn, các đoạn mã tiêu biểu cho những mẫu thiết kế chính, giao diện người dùng và cấu hình triển khai.

5.1 Công nghệ và quy mô

Thành phần	Công nghệ
Ngôn ngữ	TypeScript 5 (API, web, dự án sinh ra)
API	NestJS 10, Socket.IO, Prisma 5, BullMQ 5, pdfkit, TypeScript compiler API
Cơ sở dữ liệu, hàng đợi	PostgreSQL 16 (pgvector), Redis 7
Web	React 18, Vite, Ant Design, TanStack Query, Zustand, Recharts
Kiểm thử	Jest + ts-jest (API), Vitest (web), Playwright (giao diện đầu-cuối)
LLM	Nhà cung cấp tương thích OpenAI (9Router, OpenAI, OpenRouter, Groq, DeepSeek, Gemini, Ollama) hoặc Anthropic, cấu hình trong ứng dụng; `NINE_ROUTER_URL` là cổng mặc định
Triển khai	Docker (Node 20, nginx 1.27), Docker Compose
Tổ chức mã	Monorepo pnpm + Turborepo: `apps/api`, `apps/web`, `packages/shared-types`

Bảng sau cho biết quy mô mã nguồn tại thời điểm viết luận văn, được đếm tự động và không tính các tệp kiểm thử.

Phần	Tệp	Dòng
API (`apps/api/src`)	342	28 753
trong đó tầng miền (`src/domain`)	106	7 226
Web (`apps/web/src`)	76	8 715
Bộ kiểm thử API (Jest)	50	—

API gồm 26 mô-đun NestJS và 295 lớp được xuất (`export class`).

5.2 Tầng miền

Tầng miền được chia theo khái niệm thành các thư mục: `entities` (Project, Task, Milestone, Agent, v.v.), `value-objects` (Money, ProjectStatus, v.v.), `repositories` (giao diện), `events` (sự kiện miền và `EventBus`), `agent` (máy trạng thái, bộ phát sự kiện, giao diện chiến lược, học), `workflow` (đồ thị, bộ giải phụ thuộc, phiên bản ngữ nghĩa, memento), `security`, `audit`, `integration`, `reporting`, `analytics`, `estimation`, `quality`, `collaboration`.

Mỗi thực thể tự bảo vệ các bất biến của mình. Chẳng hạn, `Task.transitionTo` chỉ cho phép các chuyển trạng thái hợp lệ (todo → in_progress → review → done, và review → in_progress khi QA từ chối), còn `Money` không chấp nhận giá trị âm. Kiểm thử tầng miền từng phát hiện `Task.getCostVariance()` ném ngoại lệ với mọi công việc chưa dùng hết ngân sách, vì chênh lệch âm không biểu diễn được bằng `Money`; hàm này đã được sửa để trả về số đô-la có dấu.

```
        // apps/api/src/domain/repositories/IProjectRepository.ts (rút gọn)
        export interface IProjectRepository {
          findById(id: string): Promise<ProjectDTO | null>;
          findAll(options?: { orderBy?: 'createdAt' | 'updatedAt'; order?: 'asc'
| 'desc' }): Promise<ProjectDTO[]>;
          create(data: CreateProjectData): Promise<ProjectDTO>;
          update(id: string, data: UpdateProjectData): Promise<ProjectDTO>;
        }
```

Lớp hiện thực `PrismaProjectRepository` nằm trong `src/infrastructure/persistence` và chỉ truy vấn các cột mà DTO cần. Kiểm thử tải từng phát hiện rằng truy vấn danh sách dự án tải cả ngữ cảnh JSON của lượt chạy mới nhất rồi bỏ đi không dùng; sau khi sửa, thời gian truy vấn giảm từ 46 ms xuống 9 ms.

5.3 Tác tử

```
        // apps/api/src/modules/agents/base.agent.ts (rút gọn)
        async execute(input: AgentContext): Promise<AgentResult<TOutput>> {
          const context = this.prepareContext(input);          // loại công việc
theo RoleProfile
          const lifecycle = new AgentStateMachine(AgentState.IDLE); // một máy
trạng thái mỗi lần chạy
          this.running.add(lifecycle);
          try {
            await this.publish(new TaskAssignedEvent(/* … */));
            this.transition(lifecycle, AgentState.THINKING, 'Starting thought
process');
            const thought = await this.thinkingStrategy.think(context);   //
Strategy
            this.transition(lifecycle, AgentState.ACTING, 'Executing action');
            const action = await this.actionStrategy.act(context, thought); //
Strategy
            this.transition(lifecycle, AgentState.COMPLETED, 'Done');
            // lưu ký ức, phát AgentCompletedTaskEvent, quay về IDLE …
          } catch (error) {
            this.transition(lifecycle, AgentState.ERROR, (error as
Error).message);
            // …
          } finally {
            this.running.delete(lifecycle);
          }
        }
```

Mỗi vai trò có một tệp hồ sơ trong `modules/agents/profiles` và hai chiến lược trong `modules/agents/strategies`. Danh sách tác tử được NestJS tiêm vào qua token `AGENTS`; `AgentOrchestrator` dựng một bảng tra cứu theo vai trò và báo lỗi nếu có hai tác tử trùng vai trò. Như vậy, để thêm một vai trò thứ chín, chỉ cần thêm một giá trị `AgentRole`, một hồ sơ, hai chiến lược và một dòng đăng ký provider; bộ điều phối và bộ máy quy trình giữ nguyên.

Các chiến lược hành động của BA, Architect, DEV và QC ghi sản phẩm thông qua `ProjectWorkspace`. `OoadArtifactService` (mẫu Facade) dựng tài liệu và khung mã từ các mẫu Handlebars; nội dung do LLM trả về được tách thành các khối `<CODE filepath="...">`, và từng đường dẫn được kiểm tra trước khi ghi.

5.4 Bộ máy quy trình

`WorkflowEngine` giữ một bảng ánh xạ từ loại nút (`type`) sang `NodeExecutionStrategy`. Khi có Redis, mỗi nút là một công việc BullMQ (tối đa 5 công việc chạy đồng thời); khi không có Redis, nút được thực thi trực tiếp. Khi mỗi nút kết thúc, `handleJobCompletion` ghi lại bước kèm thời lượng, chụp memento và chọn cạnh kế

tiếp. Nếu nút vẫn thất bại sau các lần thử lại, kết quả được chuyển thành `failure` để lượt chạy đi theo cạnh `onFail` hoặc kết thúc ở trạng thái `failed`.

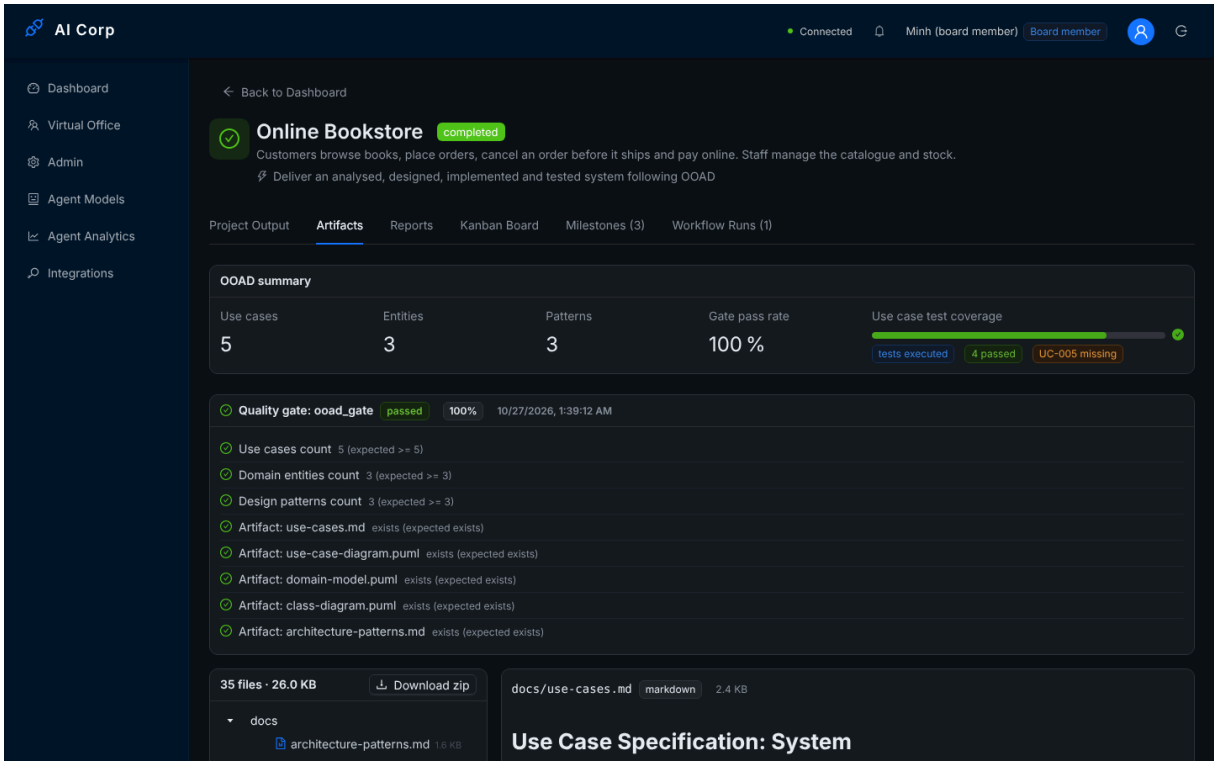
Các mẫu quy trình là lớp con của `WorkflowTemplate`, có danh sách tham số định kiểu (số, chuỗi, enum, boolean) được kiểm tra trước khi dựng định nghĩa quy trình. Ví dụ, mẫu *backend REST API* có tham số `database` kiểu enum; giá trị của tham số này được truyền vào lời nhắc thiết kế của Architect như một ràng buộc của nút, nhờ đó các Repository được thiết kế phù hợp với cơ sở dữ liệu đã chọn.

5.5 Tính năng doanh nghiệp

- **Phân quyền:** controller được gắn `@RequirePermission(Permission.PROJECT_CREATE)`; quyền của từng vai trò được lấy từ gói `shared-types` để web và API dùng chung một định nghĩa. Giao diện ẩn các hành động mà người dùng không có quyền (ví dụ nút *New Project* đối với người xem), nhưng API vẫn kiểm tra quyền một cách độc lập.
- **Kiểm toán:** route được gắn `@Audited({ action: 'approval.approve', resourceType: 'Approval', idParam: 'id' })`; interceptor ghi kết quả *ok* hoặc *denied* kèm địa chỉ IP. Trang nhật ký có bộ lọc, phân trang và chức năng xuất CSV, trong đó các ô bắt đầu bằng `=`, `+`, `-`, `@` được thoát để chống chèn công thức.
- **Xuất báo cáo:** hỗ trợ các định dạng JSON, CSV và PDF (dùng pdfkit với phông chữ hỗ trợ tiếng Việt có dấu); báo cáo tuần được ghi vào thư mục `reports/` của dự án thay vì gửi qua email.
- **Nhà cung cấp LLM:** trên trang *LLM Providers* (chỉ dành cho quản trị viên), người dùng thêm nhà cung cấp từ các mẫu có sẵn, kiểm tra kết nối, lấy danh sách mô hình từ nhà cung cấp hoặc thêm mô hình theo mã; trên trang *Agent Models*, người dùng chọn mô hình cho từng vai trò, với danh sách mô hình được nhóm theo nhà cung cấp. Mọi thay đổi đều được ghi kiểm toán, trong đó khoá đã được che.
- **Điều khiển lượt chạy:** phần đầu trang chi tiết dự án có các nút tạm dừng, tiếp tục, huỷ, bắt đầu quy trình và điều chỉnh ngân sách; một biểu ngữ thông báo khi ngân sách đã hết hoặc khi quy trình chưa bắt đầu. Yêu cầu phê duyệt được trả lời trong một hộp thoại có phần tóm tắt dễ đọc, và người dùng bắt buộc phải nêu lý do khi từ chối.

5.6 Giao diện người dùng

Ứng dụng web (với nhãn giao diện bằng tiếng Anh) gồm các màn hình: Bảng điều khiển (danh sách dự án có tìm kiếm, lọc, sắp xếp, tiến độ thực tế của lượt chạy và chi phí), Văn phòng ảo (trạng thái của tám tác tử theo thời gian thực và Phòng họp), Chi tiết dự án (kết quả, Sản phẩm, Báo cáo, Kanban, Mốc, Lượt chạy), Phân tích tác tử, Mô hình tác tử, Nhà cung cấp LLM, Người dùng và vai trò, Nhật ký kiểm toán, và Tích hợp. Tab *Artifacts* (Sản phẩm) hiển thị tóm tắt OOAD, kết quả cổng chất lượng, cây tệp của dự án, trình xem Markdown, mã nguồn và sơ đồ PlantUML (được render bởi máy chủ PlantUML nội bộ), cùng nút tải toàn bộ dự án dưới dạng tệp zip.



Hình 5.1. *Tab Sản phẩm của dự án Online Bookstore*

Trong quá trình chuẩn bị trình diễn, hai lỗi hiển thị đã được phát hiện và sửa: thẻ dự án luôn hiển thị “Starting… 0%” vì dữ liệu danh sách dự án không kèm thông tin lượt chạy, và tổng chi phí hiển thị số thực chưa được làm tròn. Đợt rà soát giao diện sau đó phát hiện thêm một lỗi nghiêm trọng hơn: trang dự án gửi yêu cầu tham gia phòng WebSocket với payload khác với định dạng mà API mong đợi, nên các sự kiện thời gian thực về công việc, quy trình và ngân sách không bao giờ đến được giao diện. Sau khi sửa, trang tham gia đúng phòng, tự tham gia lại sau khi kết nối lại và loại bỏ các sự kiện bị trùng.

5.7 Cấu hình triển khai

- `apps/api/Dockerfile` gồm hai giai đoạn. Giai đoạn build cài đặt phụ thuộc, sinh Prisma client, build API và script seed. Giai đoạn chạy dùng `tini`, chạy dưới người dùng `node`, có phông DejaVu cho việc xuất PDF và có HEALTHCHECK gọi `/api/health/live`. Tệp `docker-entrypoint.sh` chạy `prisma migrate deploy` rồi khởi động API.
- `apps/web/Dockerfile` build ứng dụng với `VITE_API_URL=/api`, `VITE_WS_URL=/`, `VITE_PLANTUML_SERVER=/plantuml`, để mọi lời gọi đều cùng nguồn gốc (same-origin) thông qua nginx.
- `docker-compose.prod.yml` và `.env.production.example` khai báo sáu dịch vụ (trong đó một dịch vụ chỉ chạy ở hồ sơ demo), ba volume, và chỉ công bố cổng web. Các biến môi trường mới gồm `LLM_SECRET_KEY` (khoá dùng để mã hoá khoá của nhà cung cấp; nếu đổi khoá này thì phải nhập lại khoá của các nhà cung cấp), `SIGNUP_ROLE`, `ALLOW_REGISTRATION` và `JWT_EXPIRATION`.
- Để cài đặt mới từ mã nguồn, trường `packageManager` cố định phiên bản pnpm 8.15.9 (pnpm 8.0 bị lỗi trên Node 20 trở lên); cấu hình Turborepo được viết theo cú pháp của phiên bản đã cài, với gói `shared-types` được build trước; và tệp build-info của `shared-types` được đặt trong thư mục `dist` để một bản clone mới luôn build ra mã.

Quá trình chuẩn bị triển khai đã phát hiện bốn lỗi khiến cấu hình cũ không thể chạy. Thứ nhất, thư mục migration chỉ chứa lược đồ ban đầu (thiếu 8 bảng và nhiều cột, do quá trình phát triển dùng `prisma db push`). Thứ hai, tệp khoá migration nằm sai vị trí. Thứ ba, cấu hình nginx bỏ mất tiền tố `/api`, nên mọi lời gọi API đều trả về 404 và WebSocket không được chuyển tiếp. Thứ tư, Dockerfile của API gọi điểm kiểm tra sức khoẻ ở đường dẫn sai và chạy một tệp không tồn tại. Cả bốn lỗi đã được sửa và kiểm chứng (mục 6.3).

Chương 6
KIỂM THỬ VÀ ĐÁNH GIÁ

Chương này trình bày phương pháp kiểm chứng nền tảng và các kết quả đo được. Mọi số liệu đều được lấy từ các lần chạy có lưu vết trong kho mã; các lệnh để tái tạo kết quả được trình bày ở Phụ lục C và D.

6.1 Chiến lược kiểm thử

Hoạt động kiểm thử được tổ chức theo mô hình kim tự tháp kiểm thử [52], gồm bốn mức:

1. **Kiểm thử đơn vị** cho tầng miền (hàm thuần, không cần giả lập) và cho các dịch vụ ứng dụng (dùng Repository trong bộ nhớ, LLM giả lập và sandbox giả lập).
2. **Kiểm thử tích hợp HTTP**: chạy một ứng dụng Nest thật với các guard, interceptor và controller thật, dữ liệu lưu trong bộ nhớ, và gọi qua giao thức HTTP.
3. **Kiểm thử đầu-cuối** trên hạ tầng thật, gồm API đã build, PostgreSQL, Redis, nginx, một LLM kịch bản tương thích OpenAI và trình duyệt Playwright cho giao diện. Mỗi lần chạy có một tệp điều khiển in ra một dòng cho mỗi kiểm tra và trả về mã lỗi khác 0 nếu có kiểm tra thất bại.
4. **Kiểm thử tải** với ngân sách thời gian phản hồi cho từng endpoint.

Việc sử dụng LLM kịch bản là một lựa chọn có chủ đích. LLM này trả lời theo nội dung lời nhắc (ví dụ, dự án có tên bắt đầu bằng “Legacy” sẽ nhận mã lần đầu vi phạm phân lớp), có độ trễ tăng theo độ dài lời nhắc và dùng một tên mô hình có bảng giá. Nhờ đó, thời lượng và chi phí thay đổi giữa các dự án, đủ để huấn luyện mô hình dự báo, trong khi mọi lần chạy vẫn cho cùng một kết quả.

6.2 Kiểm thử tự động và độ bao phủ

Bộ kiểm thử	**Kết quả**
API (Jest)	50 bộ, **707 đạt**, 15 bỏ qua (cần Docker daemon cho sandbox), 0 thất bại
Web (Vitest)	44 đạt
Biên dịch	`nest build` 0 lỗi; `tsc --noEmit` của web 0 lỗi; `vite build` thành công

Ngưỡng độ bao phủ được đặt trong cấu hình Jest, do đó lệnh `npm run test:cov` sẽ thất bại nếu không đạt ngưỡng:

Phạm vi	Dòng	Lệnh	Hàm	Nhánh	Ngưỡng (dòng/lệnh/hàm/nhánh)
Tầng miền (`src/domain`)	**97,1%**	**96,2%**	**97,0%**	**88,1%**	90 / 90 / 90 / 85
Toàn API	77,8%	77,7%	74,4%	63,5%	sàn 70 / 70 / 64 / 55

Ngưỡng 90% được áp dụng cho tầng miền thay vì cho toàn bộ API, vì tầng miền chứa các quy tắc nghiệp vụ, còn controller và phần kết nối với NestJS chủ yếu là mã khung; nếu áp ngưỡng 90% cho các phần này thì thực chất là kiểm thử chính framework. Toàn bộ API vẫn có ngưỡng sàn để độ bao phủ không bị giảm theo thời gian.

6.3 Kiểm thử đầu-cuối

Bảng sau tóm tắt ba lần chạy đầu-cuối chính; mỗi lần đều bắt đầu từ một cơ sở dữ liệu trống.

Lần chạy	Nội dung	Kết quả
Triển khai và trình diễn	Cấu hình production qua nginx, dữ liệu trình diễn, phân quyền, phê duyệt, xuất PDF, phân tích, kiểm toán, nhà cung cấp LLM	**24/24**
Phân tích, dự báo, chất lượng mã	8 dự án chạy qua toàn bộ quy trình OOAD, sau đó 3 lượt chạy khi mô hình của Marketing ngừng hoạt động; dùng bộ nhận webhook thật	**21/21**
Bảo mật, kiểm toán, tích hợp	Vai trò và quyền, nhật ký kiểm toán, khoá API, giới hạn tần suất, webhook ký HMAC, xuất dữ liệu, báo cáo	**44/44**

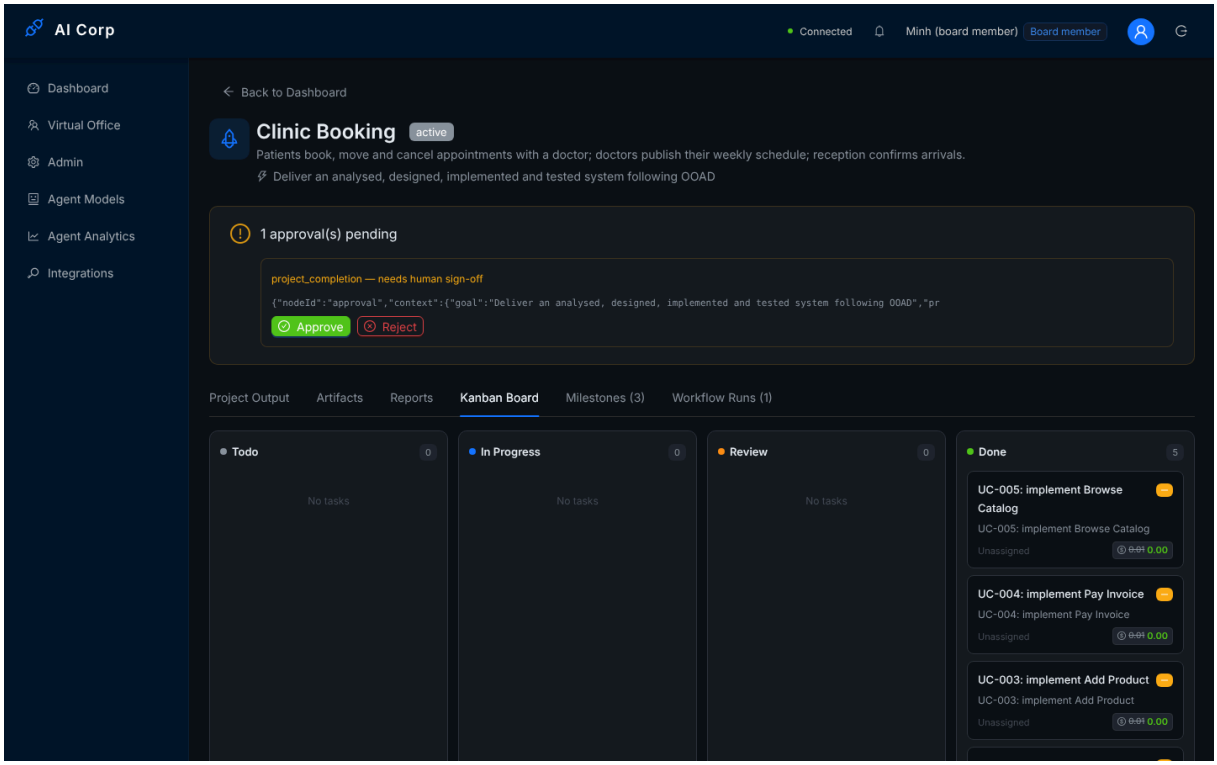
Ngoài ra còn có các lần chạy riêng cho bộ máy quy trình (quy trình con, ghim phiên bản, ảnh chụp và khôi phục, nhánh song song) và cho hệ thống tác tử (lưu sản phẩm OOAD, thực thi kiểm thử của QC, cơ chế học, lập trình cặp).

Lần chạy triển khai kiểm tra những điều mà kiểm thử đơn vị không phát hiện được:

- Lệnh `prisma migrate deploy` dựng đúng lược đồ trên cơ sở dữ liệu trống (ba migration, trong đó có migration cho bảng nhà cung cấp LLM), và lược đồ thu

được không sai lệch so với mô hình (`prisma migrate diff --exit-code` trả về 0).

- API ở chế độ production từ chối khởi động khi cấu hình sai (thiếu `JWT_SECRET` và `CORS_ORIGIN`, khoá LLM là `CHANGE_ME`): tiến trình thoát với mã 1 và thông báo liệt kê đủ ba lỗi.
- Khi truy cập qua nginx, điểm kiểm tra sức khoẻ báo cơ sở dữ liệu, Redis và cổng LLM đều ở trạng thái *healthy*. Khi dừng PostgreSQL, `/api/health` trả về mã 503 trong khi `/api/health/live` vẫn trả về 200; khi PostgreSQL khởi động lại, kết quả trở về 200.
- Swagger bị tắt khi `ENABLE_SWAGGER=false`; CORS chỉ trả về header cho nguồn gốc đã được cấu hình.
- Kết nối WebSocket qua nginx chấp nhận token đăng nhập hợp lệ và ngắt kết nối khi dùng token giả.
- Người xem không tạo được dự án, không phê duyệt được qua REST và cũng không phê duyệt được qua WebSocket (lỗ hổng này được phát hiện và sửa trong chính lần chạy này, xem mục 6.9).
- Một yêu cầu phê duyệt chỉ được trả lời một lần; câu trả lời thứ hai nhận mã 409. Tài khoản tự đăng ký nhận vai trò người xem. Cổng LLM trong cấu hình được nhập thành nhà cung cấp đầu tiên (19 mô hình), và API không trả về khoá của nhà cung cấp này.
- Sau khi thành viên hội đồng quản trị phê duyệt, CEO đóng dự án; nhật ký kiểm toán ghi nhận các sự kiện `auth.login`, `project.create`, `approval.requested`, `approval.approve`, `access.denied`, `quality.gate.evaluated`, `workflow.completed`.



Hình 6.1. *Phê duyệt đang chờ trên dự án Clinic Booking*

6.4 Thời gian xử lý của một nút quy trình

Yêu cầu phi chức năng đặt ra là mỗi nút quy trình được xử lý trong vòng 2 giây, không tính độ trễ của LLM, vì độ trễ này phụ thuộc vào nhà cung cấp và nằm ngoài tầm kiểm soát của nền tảng. Bảng sau trình bày thời lượng của 43 bước tác tử trong lần chạy triển khai. LLM kịch bản trả lời sau 20 ms cộng thêm 1 ms cho mỗi 25 ký tự lời nhắc, nên các con số dưới đây vẫn bao gồm một phần độ trễ giả lập.

Vai trò	Số bước	p50 (ms)	p95 (ms)	Lớn nhất (ms)
CEO	10	75	100	100
BA	5	107	121	121
Architect	5	582	607	607
PM	4	105	110	110
DEV	5	614	632	632
QC	5	1 642	1 726	1 726
QA	5	259	279	279
Marketing	4	75	81	81
Tất cả	**43**	**110**	**1 642**	p99: **1 726**

Mọi nút đều được xử lý dưới 2 giây. Kết quả gần như trùng với lần đo trước khi bộ máy được bổ sung bước kiểm tra ngân sách trước mỗi nút (p50 108 ms, p95 1 687 ms), cho thấy bước kiểm tra này không làm chậm hệ thống đáng kể. Nút chậm nhất là QC, vì tác tử này thực sự chạy bộ kiểm thử Jest của dự án sinh ra; Architect và DEV chậm hơn các vai trò khác do phải ghi nhiều tệp và chạy phân tích tĩnh.

6.5 Hiệu năng dưới tải

Kiểm thử tải sử dụng một script không phụ thuộc thư viện ngoài, mô phỏng người dùng bảng điều khiển (xem danh sách và chi tiết dự án, công việc, báo cáo chi phí, phân tích tác tử, dự báo, chất lượng mã). Mỗi endpoint có ngân sách thời gian phản hồi p50/p95/p99: mặc định là 100/500/1000 ms; 300/1500/3000 ms cho phân tích, dự báo và chất lượng mã; 500/2000/3000 ms cho tạo dự án. Tỉ lệ lỗi tối đa cho phép là 1%. Môi trường kiểm thử gồm một tiến trình API ở chế độ production, PostgreSQL và Redis, chạy trên một máy 4 lõi dùng chung với bộ sinh tải và LLM kịch bản. Cơ sở dữ liệu bắt đầu từ dữ liệu trình diễn (5 dự án) và tăng dần khi các kịch bản tạo thêm dự án.

Kịch bản	Yêu cầu/giây	Lỗi	Kết quả theo ngân sách
1 người dùng	184,7	0%	Mọi endpoint trong ngân sách; p50 từ 3 đến 7 ms
20 người dùng, chỉ đọc	**604,8**	0%	Mọi endpoint trong ngân sách; p95 cao nhất 84 ms
20 người dùng, đồng thời tạo 111 dự án	430,7	0%	Mọi endpoint trong ngân sách; tạo dự án p50 107 ms, p99 177 ms
50 người dùng, tạo 48 dự án	407,2	0%	Bão hoà: p50 77–312 ms, năm endpoint vượt ngân sách p50 100 ms; mọi p99 dưới 0,7 s

Với 50 người dùng, một tiến trình API đã bão hoà ở mức khoảng 407 yêu cầu/giây. Theo định luật Little [41], mỗi yêu cầu khi đó phải chờ khoảng 50 / 407 ≈ 123 ms; đây là giới hạn của một tiến trình đơn lẻ, còn bản thân các truy vấn vẫn nhanh. Vì API không lưu trạng thái (các lượt chạy được điều phối qua BullMQ và PostgreSQL), có thể mở rộng hệ thống bằng cách chạy thêm tiến trình.

Lần đo trước, trên một cơ sở dữ liệu có khoảng 120 dự án, cho kết quả 201,8 yêu cầu/giây với 20 người dùng và bão hoà ở mức 178 yêu cầu/giây. Hai lần đo không so sánh trực tiếp được với nhau vì kích thước dữ liệu khác nhau, do đó luận văn không coi chênh lệch giữa chúng là mức cải thiện. Điểm có thể so sánh là kết quả theo ngân

sách: trong cả hai lần đo, với 20 người dùng, mọi endpoint đều nằm trong hoặc sát ngân sách và không phát sinh lỗi. Lần đo trước đã phát hiện ba lỗi hiệu năng (mục 6.9); sau khi sửa, thời gian p50 của endpoint chất lượng mã giảm từ 3,1 s xuống 196 ms trong cùng kịch bản.

6.6 Độ chính xác của dự báo

Sau khi 8 dự án hoàn tất, mọi vai trò đều có đủ mẫu để dùng mô hình hồi quy. Bảng sau trình bày sai số leave-one-out của dự báo thời gian tác tử.

Vai trò	**MAPE (LOO)**
Architect	3%
CEO	5%
BA	11%
DEV	30%

Dự báo tại thời điểm bắt đầu của 7 dự án (không tính dự án đầu tiên vì chưa có dữ liệu lịch sử) có sai số tuyệt đối trung bình là 21% về thời gian và 20% về chi phí. Phần lớn sai số đến từ một dự án mà QA từ chối mã và DEV phải làm lại (chi phí vượt dự báo 84%); các dự án còn lại có sai lệch từ 0 đến 10%. Kết quả này phù hợp với bản chất của bài toán: việc phải làm lại do bị từ chối không thể biết trước khi dự án bắt đầu. Vì vậy, nền tảng lưu cả dự báo tại thời điểm bắt đầu và dự báo khi PM lập kế hoạch, đồng thời luôn hiển thị dự báo bên cạnh kết quả thực tế.

6.7 Dự đoán chất lượng mã

Trường hợp	**Diễn biến**
Legacy Billing	Mã lần đầu của DEV nhập tầng hạ tầng từ tầng miền, chứa khoá viết cứng, gọi `eval` và dồn các quy tắc vào một phương thức dài. Điểm 77 và có vi phạm phân lớp nên dự đoán là *fail*; DEV sửa một vòng, điểm tăng lên 100, bản sửa được giữ lại và QA duyệt
Unsafe Portal	Vòng sửa không cải thiện điểm nên bản gốc được giữ lại; QA phát hiện `eval` qua báo cáo phân tích và trả lời FAIL; quy trình quay lại DEV và lần thứ hai đạt
Toàn bộ	Trong 9 lần QA rà soát, 8 lần dự đoán *pass* và được duyệt, 1 lần dự đoán *fail* và bị từ chối

Độ chính xác 100% trên 9 lần rà soát cần được diễn giải thận trọng, vì với LLM kịch bản, các lỗi được cài sẵn đúng loại mà bộ phân tích có thể phát hiện. Con số này cho thấy chuỗi dự đoán, sửa lỗi và đối chiếu hoạt động đúng, nhưng chưa chứng minh được độ chính xác khi dùng LLM thật (mục 7.4).

6.8 Nghiên cứu tình huống

Năm dự án trình diễn được tạo thông qua API công khai giống như một người dùng thật; mỗi dự án minh hoạ một khía cạnh của nền tảng.

Dự án	Mẫu quy trình	Kết quả
Online Bookstore	Dự án phần mềm OOAD	5 use case, 3 thực thể, 3 mẫu thiết kế, đạt cổng OOAD 100%; 35 tệp; 4 kiểm thử chấp nhận chạy đạt; độ bao phủ use case 80%, UC-005 được báo thiếu kiểm thử; có thông báo marketing; hoàn tất
Legacy Billing	Dự án phần mềm OOAD	Phân tích tĩnh phát hiện vi phạm, DEV tự sửa (điểm từ 77 lên 100) trước khi QA rà soát; hoàn tất
Unsafe Portal	Dự án phần mềm OOAD	QA từ chối, quy trình quay lại DEV (2 bước DEV, 2 lần rà soát QA); hoàn tất
Library Loans	Chỉ phân tích và thiết kế	Use case, mô hình miền và sơ đồ vượt qua cổng OOAD; hoàn tất
Clinic Booking	Dự án phần mềm OOAD, có phê duyệt	Dừng ở bước phê duyệt; người xem không phê duyệt được; sau khi thành viên hội đồng quản trị phê duyệt, CEO đóng dự án

Với LLM kịch bản, cả năm dự án được tạo và chạy xong trong khoảng 25 giây.

6.9 Lỗi phát hiện nhờ kiểm thử và rà soát

Việc kiểm thử trên hạ tầng thật đã giúp phát hiện và sửa nhiều lỗi. Ngoài kiểm thử tự động, lỗi còn được phát hiện từ hai nguồn khác: một đợt rà soát mã có hệ thống đối với phần giao diện và logic, trong đó từng màn hình được đọc và đối chiếu với API mà

màn hình đó gọi; và việc cài đặt mới từ mã nguồn trên một máy khác. Bảng sau tóm tắt các lỗi theo nhóm; danh sách đầy đủ kèm cách sửa nằm trong kế hoạch dự án và lịch sử thay đổi của kho mã.

Nhóm	Ví dụ tiêu biểu	Kiểm thử	Rà soát	Cài đặt mới	Tổng
Không khởi động / không chạy được	API không khởi động vì hai chiến lược có tham số hàm tạo mà DI không giải được; migration thiếu 8 bảng; nginx bỏ tiền tố `/api`; pnpm 8.0 lỗi trên Node 20; một bản clone mới không build ra `shared-types`; script dữ liệu trình diễn hỏng khi đăng ký mặc định thành người xem	6	0	4	10
Bảo mật	Duyệt qua WebSocket bỏ qua quyền và kiểm toán; ba giá trị dự phòng khác nhau cho khoá JWT; tự đăng ký nhận quyền thành viên hội đồng; một phê duyệt trả lời được hai lần; sửa công việc không kiểm tra chủ dự án	3	3	0	6
Thiết kế OOAD không được dùng	4/8 tác tử vẫn chạy luồng cũ nên chiến lược là mã chết; cổng chất lượng không được đăng ký; sản phẩm của tác tử mất sau mỗi lệnh sandbox; quy trình lập kế hoạch trước khi có yêu cầu	6	0	0	6
Tính đúng của quy trình và trạng thái	Một máy trạng thái cho mỗi vai trò; nút thất bại để lượt chạy `running` mãi; *Pause* chỉ đổi trạng thái dự án; *Resume* ở bước phê duyệt tạo phê duyệt trùng; dự án đã huỷ bị đánh dấu hoàn tất; lý do từ chối không đến được tác tử	8	6	0	14
Chi phí và sự kiện	Chi phí LLM của 4 vai trò không được ghi; nút vẫn bắt đầu khi ngân sách đã hết; cộng chi phí kiểu đọc rồi ghi bị mất cập nhật khi đồng thời; không có sự kiện `approval.requested` và `workflow.failed`	4	3	0	7
Hiệu năng	Phân tích lại mọi tệp ở mỗi yêu cầu (3,1 s); danh sách dự án tải JSON không dùng; bốn truy vấn tuần tự	3	0	0	3
Tài liệu và giao diện	12/39 sơ đồ UML không render được; thẻ dự án luôn “0%”; trang dự án tham gia phòng WebSocket sai payload nên không nhận sự kiện thời gian thực	3	1	0	4
Tổng		**33**	**13**	**4**	**50**

Cả mười ba lỗi do rà soát phát hiện đều nằm trên những luồng xử lý mà kiểm thử tự động chưa đi qua. Các kiểm thử đầu-cuối chạy quy trình từ đầu đến cuối, nhưng không tạm dừng rồi tiếp tục, không trả lời một phê duyệt hai lần, và không thực hiện hai lần cộng chi phí cùng lúc. Sau khi sửa, mỗi lỗi đều có một kiểm thử đơn vị riêng (`run-control.spec.ts`, `approval-and-signup.spec.ts`), và ba hành vi được bổ sung vào

lần chạy đầu-cuối triển khai: phê duyệt chỉ được trả lời một lần, tài khoản đăng ký chỉ nhận quyền xem, và khoá của nhà cung cấp không bị trả về. Chính lần chạy lại này đã phát hiện script tạo dữ liệu trình diễn vẫn giả định rằng tài khoản tự đăng ký có vai trò thành viên hội đồng quản trị.

Lỗ hổng phê duyệt qua WebSocket là trường hợp đáng chú ý, vì mọi kiểm thử trước đó đều đi qua REST. Lỗ hổng chỉ bộc lộ khi kiểm thử đi theo đúng luồng mà giao diện sử dụng: người xem nhận mã 403 khi gọi `POST /api/approvals/:id/approve`, nhưng khi gửi thông điệp `human:approval_response` qua WebSocket thì phê duyệt vẫn được chấp nhận, dự án được đóng và không có mục kiểm toán nào được ghi. Bài học thiết kế rút ra là mọi luồng ghi dữ liệu phải đi qua cùng một điểm kiểm soát. Sau khi sửa, kênh WebSocket từ chối thông điệp này, giao diện chuyển sang dùng REST, và lần chạy đầu-cuối có một kiểm tra riêng cho trường hợp này.

Chương 7
THẢO LUẬN

Dựa trên kết quả ở chương 6, chương này trả lời ba câu hỏi nghiên cứu đã nêu ở chương 1, sau đó phân tích các yếu tố đe doạ tính hợp lệ của đánh giá và các hạn chế của nền tảng.

7.1 RQ1: OOAD có cải thiện kiến trúc của hệ đa tác tử dùng LLM không?

Câu trả lời là có, kèm theo một điều kiện quan trọng được nêu ở cuối mục này. Bằng chứng được xem xét theo ba thuộc tính: tính mô-đun, khả năng mở rộng và khả năng kiểm thử.

Tính mô-đun. Tầng miền gồm 106 tệp (7 226 dòng) và không nhập NestJS, Prisma hay bất kỳ thư viện LLM nào; mọi phụ thuộc vào hạ tầng đều đi qua giao diện Repository hoặc các cổng (port) như `ICodeAnalyzer`. Các mô-đun giao tiếp với nhau bằng sự kiện miền qua `EventBus`, nhờ đó kiểm toán, webhook và phân tích hiệu năng được bổ sung mà không phải sửa nơi phát sự kiện. Trong quá trình phát triển, nhiều thay đổi lớn chỉ tác động đến một vị trí trong mã. Việc ghi chi phí cho cả tám vai trò được giải quyết bằng một Decorator bọc quanh nhà cung cấp LLM. Việc đóng lỗ hổng phê duyệt qua WebSocket chỉ cần xoá một luồng ghi và chuyển giao diện sang REST, trong khi phần kiểm tra quyền và kiểm toán giữ nguyên. Việc chặn chi tiêu sau khi hết ngân sách là một bước kiểm tra trong bộ máy trước mỗi nút, và các chiến lược nút không cần biết đến bước này.

Khả năng mở rộng. Để thêm một vai trò tác tử, chỉ cần thêm một giá trị `AgentRole`, một `RoleProfile`, hai lớp chiến lược và một dòng đăng ký provider; `AgentOrchestrator` và `WorkflowEngine` giữ nguyên (mục 5.3). Để thêm một loại nút quy trình, chỉ cần thêm một `NodeExecutionStrategy` tự đăng ký; quy trình con, cổng chất lượng, phê duyệt và ba mẫu cộng tác đều được bổ sung theo cách này. Tương tự, thêm một định dạng xuất, một quy tắc phát hiện bất thường hay một tín hiệu chất lượng đều chỉ là thêm một lớp vào sổ đăng ký. Ví dụ rõ nhất là tính năng quản lý nhà cung cấp LLM trong ứng dụng (mục 4.3.3), được bổ sung sau khi các tác tử đã hoàn chỉnh. Tính năng này gồm một aggregate mới, một cổng mã hoá và một chiến lược cho mỗi loại nhà cung cấp, nhưng không tác tử, chiến lược tác tử hay decorator ghi chi phí nào phải sửa, vì tất cả chỉ phụ thuộc vào giao diện `ILLMProvider`.

Khả năng kiểm thử. Vì tầng miền độc lập với framework, các quy tắc nghiệp vụ được kiểm thử dưới dạng hàm thuần mà không cần giả lập, và tầng miền đạt độ bao phủ 97,1% số dòng và 88,1% số nhánh (mục 6.2). Các chiến lược tác tử được kiểm thử với LLM và sandbox giả lập thông qua giao diện; bộ máy quy trình chạy được cả khi có và khi không có Redis. Toàn bộ API có độ bao phủ thấp hơn (77,8% số dòng) do phần mã kết nối với framework. Đây là một phân bố hợp lý, vì phần mã chứa logic quyết định được kiểm thử kỹ nhất.

Điều kiện. Một cấu trúc OOAD tốt trong mã nguồn chưa bảo đảm rằng thiết kế đó thực sự được sử dụng. Mục 6.9 cho thấy có sáu lỗi thuộc nhóm "thiết kế OOAD không được dùng": bốn trên tám tác tử vẫn chạy theo luồng cũ nên các lớp chiến lược của chúng trở thành mã chết; cổng chất lượng đã được viết nhưng không được đăng ký vào bộ máy; sản phẩm của tác tử bị mất sau mỗi lệnh chạy trong sandbox. Trong khi đó, mọi kiểm thử đơn vị của các lớp này đều đạt. Chỉ có kiểm thử đầu-cuối trên hạ tầng thật, với việc kiểm tra các kết quả quan sát được (tệp trên đĩa, mục kiểm toán, trạng thái lượt chạy), mới phát hiện ra các lỗi này. Từ đó có thể kết luận rằng OOAD cải thiện kiến trúc khi đi kèm với các kiểm thử chứng minh từng luồng thiết kế thực sự được thực thi, chẳng hạn kiểm tra rằng mỗi vai trò đều đi qua `BaseAgent.execute()` và mỗi chiến lược đã đăng ký đều được gọi. Đợt rà soát giao diện và logic (mục 6.9) bổ sung thêm một nhận xét: kiểm thử đầu-cuối chỉ chứng minh được những luồng mà nó đi qua. Mười ba lỗi nằm ở các thao tác ngoài luồng chính (tạm dừng rồi tiếp tục, trả lời phê duyệt hai lần, cộng chi phí đồng thời) và chỉ được phát hiện khi đọc từng màn hình và đối chiếu với API.

7.2 RQ2: Những mẫu thiết kế nào hiệu quả nhất cho việc điều phối?

Mức độ hiệu quả của mẫu thiết kế ở đây được đánh giá theo hai tiêu chí thực tiễn: mẫu có giữ cho các thay đổi mang tính cục bộ hay không, và mẫu có giúp phát hiện hoặc ngăn ngừa lỗi trong quá trình phát triển hay không.

1. **Template Method kết hợp Strategy cho tác tử** là mẫu có giá trị nhất. Chu trình chung (phát sự kiện, chuyển trạng thái, quản lý bộ nhớ, xử lý lỗi) được đặt ở một nơi, còn chuyên môn của từng vai trò nằm trong hai chiến lược có thể thay thế độc lập. Khi phát hiện bốn tác tử còn chạy theo luồng cũ, cách sửa là đưa chúng về cùng một khung, và mọi tính năng gắn với khung (ghi chi phí, học, phát sự kiện) tự động có hiệu lực cho cả bốn tác tử.

2. **Strategy tự đăng ký cho nút quy trình**, cùng với **Composite** cho quy trình con, cho phép bộ máy quy trình phát triển từ một chuỗi tuyến tính thành một đồ thị có nhánh song song, quy trình con, phê duyệt và cộng tác mà không phải sửa vòng lặp thực thi.
3. **State**, kèm theo một bài học về phạm vi áp dụng: máy trạng thái phải được gắn với đối tượng thực sự có vòng đời. Khi gắn một máy trạng thái cho mỗi vai trò, hai công việc chạy đồng thời làm hỏng trạng thái của nhau; khi gắn cho mỗi lần thực thi, hệ thống hoạt động đúng (mục 4.3.2). Đối với công việc và dự án, việc đặt bất biến chuyển trạng thái trong thực thể đã chặn các chuyển trạng thái Kanban không hợp lệ ngay tại tầng miền.
4. **Observer và sự kiện miền** cho các mối quan tâm cắt ngang: kiểm toán, webhook, phân tích hiệu năng và thông báo thời gian thực đều đăng ký lắng nghe sự kiện thay vì được gọi trực tiếp. Tuy nhiên, hai lỗi "thiếu sự kiện" (mục 6.9) cho thấy mặt trái của cách làm này: khi một sự kiện không được phát, mọi bên lắng nghe đều không phản ứng, và chỉ kiểm thử đầu-cuối mới phát hiện được điều đó.
5. **Memento và sổ đăng ký bất biến** cho phiên bản quy trình giúp việc khôi phục và ghim phiên bản trở nên đơn giản, vì định nghĩa quy trình mà một lượt chạy đang dùng không bao giờ bị thay đổi.
6. **Mediator** (`AgentMessageBus`) giúp các tác tử không phụ thuộc trực tiếp vào nhau khi cộng tác, đồng thời tạo ra một điểm duy nhất để giới hạn số vòng trao đổi và hiển thị hội thoại.

Các mẫu ít ảnh hưởng hơn đến việc điều phối nhưng hữu ích cho những phần còn lại gồm Decorator (ghi chi phí), Facade (một điểm duy nhất để gọi LLM), Chain of Responsibility (chuỗi guard và chuỗi rà soát) và Port/Adapter (phân tích mã). Một nhận xét chung là mẫu thiết kế phát huy hiệu quả cao nhất khi tạo ra một điểm kiểm soát duy nhất. Lỗ hổng phê duyệt qua WebSocket xuất hiện chính vì tồn tại một luồng ghi thứ hai đi vòng qua điểm kiểm soát đó. Các lỗi phát hiện trong đợt rà soát cũng có cùng dạng: nút *Pause* thay đổi trạng thái dự án mà không đi qua bộ máy quy trình; chi phí được cộng theo kiểu đọc rồi ghi ở hai nơi; và trạng thái của một yêu cầu phê duyệt được kiểm tra ở một bước nhưng lại được cập nhật ở bước khác. Trong cả ba trường hợp, cách sửa là đưa thao tác về một điểm duy nhất có tính nguyên tử: thao tác trên lượt chạy trong bộ máy quy trình, phép tăng nguyên tử trong cơ sở dữ liệu, và cập nhật có điều kiện trên trạng thái `pending`.

7.3 RQ3: Tác tử học từ kết quả như thế nào, và đo điều đó ra sao?

Nền tảng có hai dạng học, và mỗi dạng được đo lường theo một cách khác nhau.

Học để dự báo sử dụng lịch sử các lượt chạy để ước lượng thời gian và chi phí. Dạng học này đo được trực tiếp bằng sai số leave-one-out: sau tám dự án, MAPE theo vai trò dao động từ 3% (Architect) đến 30% (DEV), và dự báo tại thời điểm bắt đầu có sai số trung bình khoảng 20% (mục 6.6). Sai số lớn nhất đến từ việc làm lại sau khi QA từ chối, một sự kiện không thể biết trước khi dự án bắt đầu. Vì vậy, nền tảng hiển thị dự báo bên cạnh kết quả thực tế và không coi dự báo là một cam kết.

Học để cải thiện sản phẩm là việc chắt lọc các kết quả khách quan (cổng OOAD, kiểm thử của QC, phán quyết của QA) thành bài học và đưa các bài học đó vào lời nhắc ở những lần sau (mục 4.3.4). Các lần chạy đầu-cuối đã kiểm chứng rằng chuỗi xử lý này hoạt động đúng: kết quả được ghi cho đúng vai trò, bài học được chắt lọc sau mỗi N kết quả và xuất hiện trong lời nhắc tiếp theo. Tuy nhiên, luận văn chưa đo được mức cải thiện chất lượng nhờ cơ chế học. Với LLM kịch bản, câu trả lời không phụ thuộc vào các bài học trong lời nhắc, nên mọi phép so sánh trước và sau đều cho kết quả như nhau do cách xây dựng kịch bản. Vì vậy, luận văn không báo cáo con số cải thiện nào trong điều kiện này.

Luận văn đề xuất cách đo như sau, và đây cũng là hướng phát triển đầu tiên được nêu ở chương 8: sử dụng LLM thật với nhiệt độ cố định; chia một tập dự án cố định thành hai nhóm, một nhóm có học và một nhóm không học (tắt `LEARNING_REFINE_EVERY`); chạy mỗi nhóm nhiều lần; so sánh tỉ lệ đạt cổng OOAD ngay từ lần đầu, số lần QA từ chối, số vòng sửa và chi phí làm lại, kèm theo kiểm định thống kê. Hạ tầng cần thiết cho thí nghiệm này đã có sẵn, gồm tín hiệu kết quả, phân tích hiệu năng theo vai trò và ghi nhận chi phí của từng lời gọi.

7.4 Mối đe doạ đến tính hợp lệ của đánh giá

Phần này sử dụng cách phân loại thường gặp trong nghiên cứu thực nghiệm về kỹ nghệ phần mềm, gồm tính hợp lệ nội tại, tính hợp lệ cấu trúc và tính hợp lệ ngoại tại.

Tính hợp lệ nội tại: LLM kịch bản. Mọi lần chạy đầu-cuối đều sử dụng một máy chủ LLM kịch bản tương thích OpenAI. Máy chủ này trả lời theo nội dung lời nhắc (tên dự án, vai trò, bước), với độ trễ và giá được giả lập. Lựa chọn này giúp kết quả lặp lại được và cho phép kiểm thử chính xác các nhánh khó tái hiện với LLM thật, như mã vi

phạm phân lớp, QA từ chối hay mô hình ngừng hoạt động. Đổi lại, lựa chọn này giới hạn phạm vi những gì kết quả có thể chứng minh:

- Các kết quả về luồng điều khiển là hợp lệ: quy trình đi đúng nhánh, trạng thái được lưu, quyền được kiểm tra, chi phí được ghi nhận và lỗi được xử lý. Các kết quả này không phụ thuộc vào nội dung câu trả lời của LLM.
- Các kết quả về chất lượng nội dung chỉ mang tính minh hoạ. Độ chính xác 100% của dự đoán chất lượng mã (mục 6.7) đạt được vì các lỗi được cài sẵn đúng loại mà bộ phân tích phát hiện được. Số use case, số tệp và độ bao phủ use case của các dự án trình diễn phản ánh kịch bản đã soạn, chưa phản ánh năng lực của một mô hình thật.
- Các số liệu về thời gian bao gồm độ trễ giả lập (20 ms cộng 1 ms cho mỗi 25 ký tự lời nhắc). Với LLM thật, độ trễ của mỗi lời gọi thường tính bằng giây và chiếm phần lớn tổng thời gian; đó là lý do yêu cầu 2 giây cho mỗi nút được định nghĩa là không tính độ trễ của LLM.
- Mô hình dự báo được huấn luyện trên thời lượng và chi phí do kịch bản sinh ra, nên MAPE thấp của Architect và CEO một phần là do hành vi giả lập có tính ổn định cao. Với LLM thật, phương sai sẽ lớn hơn và sai số dự kiến sẽ cao hơn.

Tính hợp lệ cấu trúc. "Chất lượng OOAD" của dự án sinh ra được đo bằng cổng chất lượng và bộ phân tích tĩnh do chính luận văn định nghĩa: có đủ use case, thực thể và sơ đồ, không vi phạm phân lớp, và độ phức tạp nằm trong ngưỡng. Đây là các điều kiện cần nhưng chưa đủ; một thiết kế vượt qua được cổng chất lượng vẫn có thể bị chuyên gia đánh giá là chưa tốt. Tương tự, độ bao phủ mã chỉ đo số dòng được thực thi, không đo chất lượng của các khẳng định (assertion) trong kiểm thử.

Tính hợp lệ ngoại tại. Kiểm thử tải chạy một tiến trình API trên một máy 4 lõi dùng chung với bộ sinh tải; số liệu thông lượng là giới hạn dưới cho cấu hình này và không dùng để dự đoán cho một cụm máy. Các dự án sinh ra đều là ứng dụng TypeScript cỡ nhỏ (khoảng 35 tệp), nên kết quả chưa cho thấy nền tảng có xử lý được dự án lớn hay ngôn ngữ khác hay không. Luận văn cũng chưa tiến hành nghiên cứu với người dùng thật, nên chưa có kết luận về mức độ hữu ích của nền tảng đối với một nhóm phát triển.

Biện pháp giảm thiểu. Để hạn chế các yếu tố đe doạ nêu trên, luận văn chỉ báo cáo số liệu từ các lần chạy có lưu vết, tách rõ kết quả về luồng điều khiển với kết quả về nội

dung, đồng thời công bố mã nguồn của LLM kịch bản cùng các script đo để người khác có thể tái tạo kết quả hoặc thay bằng LLM thật (Phụ lục C và D).

7.5 Các hạn chế của nền tảng

- **Sandbox:** môi trường kiểm thử không có Docker daemon, nên 15 kiểm thử của sandbox container bị bỏ qua, và các lần chạy đầu-cuối thực thi kiểm thử của dự án sinh ra trong một bản sao tạm thời trên máy chủ (`SANDBOX_ALLOW_LOCAL_RUN=true`). Cách làm này an toàn trong phạm vi luận văn vì mã nguồn do LLM kịch bản viết; với LLM thật, bắt buộc phải dùng sandbox container, và luồng chạy đó chưa được kiểm chứng đầu-cuối.
- **Ngôn ngữ đích:** mẫu sản phẩm, bộ phân tích tĩnh và việc chạy kiểm thử chấp nhận hiện chỉ hỗ trợ TypeScript.
- **Con người trong vòng lặp:** phê duyệt là một nút chặn, trong đó người dùng duyệt hoặc từ chối kèm lý do, và lý do được chuyển cho các tác tử chạy sau. Nền tảng chưa hỗ trợ chỉnh sửa trực tiếp sản phẩm rồi cho quy trình tiếp tục từ bản đã sửa, chưa cho phép giao lại một yêu cầu phê duyệt cho người khác, và quy trình mặc định chưa có nhánh riêng khi phê duyệt bị từ chối.
- **Mở rộng ngang:** API không lưu trạng thái nên có thể chạy nhiều tiến trình, nhưng cấu hình triển khai hiện tại chỉ chạy một tiến trình và chưa được kiểm thử tải ở chế độ nhiều bản sao. Để hỗ trợ kết nối WebSocket với nhiều bản sao, cần bổ sung bộ chuyển (adapter) Redis cho Socket.IO.
- **Bảo mật:** các cơ chế kiểm soát (phân quyền, khoá API, giới hạn tần suất, ký HMAC, kiểm toán, kiểm tra cấu hình production) đã được kiểm thử về mặt chức năng nhưng chưa qua kiểm thử xâm nhập độc lập. Lỗ hổng WebSocket ở mục 6.9 cho thấy sự cần thiết của việc rà soát mọi luồng ghi dữ liệu.
- **Chi phí:** không nút nào được bắt đầu khi ngân sách đã hết, nhưng một nút đang chạy vẫn có thể làm vượt ngân sách một lượng bằng chi phí của chính nút đó. Nền tảng chưa có cơ chế ước lượng chi phí trước mỗi lời gọi để chặn sớm, và các lời gọi LLM quá hạn chưa bị huỷ.
- **Phiên đăng nhập:** phiên hết hạn sau 8 giờ và người dùng phải đăng nhập lại; nền tảng chưa hỗ trợ token làm mới (refresh token).

Chương 8
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

8.1 Kết luận

Luận văn đã xây dựng AI-CORP, một nền tảng đa tác tử dùng LLM mô phỏng một công ty phần mềm với tám vai trò (CEO, BA, Architect, PM, DEV, QC, QA, Marketing), và áp dụng phân tích và thiết kế hướng đối tượng ở ba cấp độ: kiến trúc của nền tảng, thiết kế của từng tác tử, và sản phẩm mà các tác tử tạo ra cho người dùng.

Về kiến trúc, nền tảng có tầng miền độc lập với framework; mỗi tác tử được xây dựng theo mẫu Template Method kết hợp hai Strategy; tác tử, công việc và dự án có máy trạng thái riêng; các mô-đun giao tiếp qua sự kiện miền; mọi phụ thuộc vào hạ tầng đi qua Repository hoặc cổng. Bộ máy quy trình hỗ trợ quy trình con, phiên bản bất biến, ảnh chụp và khôi phục, nhánh song song, phê duyệt của con người và ba mẫu cộng tác giữa các tác tử. Về sản phẩm, mỗi dự án lần lượt đi qua use case, mô hình miền, sơ đồ UML, mã nguồn phân lớp, kiểm thử chấp nhận và ma trận truy vết, với cổng chất lượng OOAD, phân tích tĩnh và một vòng tự sửa có giới hạn chi phí. Về vận hành, nền tảng có phân quyền theo vai trò, nhật ký kiểm toán, khoá API, webhook ký HMAC, xuất báo cáo, quản lý nhà cung cấp và mô hình LLM ngay trong ứng dụng, điều khiển lượt chạy có ràng buộc ngân sách, phân tích hiệu năng tác tử, dự báo thời gian và chi phí, cùng cấu hình triển khai Docker có kiểm tra cấu hình production và kiểm tra sức khoẻ.

Các kết quả chính đo được gồm:

- 707 kiểm thử API và 44 kiểm thử web đều đạt; tầng miền có độ bao phủ 97,1% số dòng và 88,1% số nhánh.
- Các lần chạy đầu-cuối trên hạ tầng thật đạt 24/24, 21/21 và 44/44 kiểm tra; cấu hình production dựng được lược đồ từ migration và lược đồ không sai lệch so với mô hình.
- Mọi nút quy trình được xử lý dưới 2 giây (p95 là 1,64 s, chưa trừ độ trễ LLM giả lập); một tiến trình API phục vụ khoảng 605 yêu cầu đọc mỗi giây với 20 người dùng và bão hoà ở khoảng 407 yêu cầu/giây với 50 người dùng, không phát sinh lỗi.
- Dự báo thời gian tác tử có MAPE leave-one-out từ 3% đến 30% tuỳ vai trò.

- Kiểm thử trên hạ tầng thật, một đợt rà soát giao diện và logic, cùng việc cài đặt mới đã phát hiện và sửa 50 lỗi, trong đó có lỗ hổng cho phép người xem phê duyệt qua WebSocket và lỗi cho phép một yêu cầu phê duyệt được trả lời hai lần.

Đối với ba câu hỏi nghiên cứu, luận văn rút ra các kết luận sau. Thứ nhất, OOAD cải thiện rõ rệt tính mô-đun, khả năng mở rộng và khả năng kiểm thử, với điều kiện có kiểm thử đầu-cuối chứng minh thiết kế thực sự được sử dụng. Thứ hai, Template Method kết hợp Strategy cho tác tử, Strategy tự đăng ký kết hợp Composite cho quy trình, và State gắn với đúng đối tượng có vòng đời là các mẫu hiệu quả nhất; nhìn chung, một mẫu thiết kế có giá trị nhất khi tạo ra một điểm kiểm soát duy nhất có tính nguyên tử. Thứ ba, việc học từ lịch sử để dự báo đo được và cho sai số chấp nhận được; còn việc học để cải thiện sản phẩm đã được hiện thực và kiểm chứng về mặt cơ chế, nhưng mức cải thiện chỉ có thể đo một cách khách quan khi dùng LLM thật trong một thí nghiệm có nhóm đối chứng.

8.2 Hướng phát triển

1. **Đánh giá với LLM thật.** Trang quản lý nhà cung cấp đã cho phép chọn mô hình thật cho từng vai trò mà không phải sửa mã, nên trở ngại chính của bước này là chi phí và thời gian chạy. Hướng phát triển này gồm việc chạy lại tập dự án đánh giá với một hoặc nhiều mô hình thật qua cổng tương thích OpenAI; đo chất lượng sản phẩm (tỉ lệ đạt cổng OOAD ngay lần đầu, số lần QA từ chối, số kiểm thử chấp nhận đạt), thời gian và chi phí thực tế; và đánh giá lại mô hình dự báo cùng bộ dự đoán chất lượng mã trên dữ liệu thu được.
2. **Thí nghiệm có đối chứng cho cơ chế học**, theo thiết kế đã nêu ở mục 7.3: hai nhóm có học và không học, lặp lại nhiều lần, kèm kiểm định thống kê.
3. **Đánh giá thiết kế bởi chuyên gia**: mời chuyên gia chấm điểm sản phẩm OOAD của các dự án sinh ra theo một thang đo, nhằm kiểm tra mức độ tương quan giữa cổng chất lượng tự động và đánh giá của con người.
4. **Đưa sandbox container vào kiểm thử thường xuyên**: chạy các kiểm thử sandbox trên một máy CI có Docker và kiểm chứng đầu-cuối luồng thực thi mã trong container.
5. **Mở rộng ngang**: bổ sung adapter Redis cho Socket.IO, chạy nhiều bản sao API phía sau nginx và kiểm thử tải với cấu hình đó.
6. **Hỗ trợ ngôn ngữ đích khác**: tách bộ phân tích tĩnh và trình chạy kiểm thử thành các adapter theo từng ngôn ngữ, đặt sau cổng `ICodeAnalyzer` đã có, bắt đầu với

Python hoặc Java.

7. **Tăng cường vai trò của con người trong vòng lặp**: cho phép người dùng sửa trực tiếp một sản phẩm (ví dụ mô hình miền) tại bước phê duyệt và để quy trình tiếp tục từ bản đã sửa, tận dụng cơ chế ảnh chụp và khôi phục sẵn có.
8. **Kiểm thử xâm nhập độc lập** cho API, WebSocket và webhook trước khi sử dụng với dữ liệu thật.
9. **Hoàn thiện cơ chế điều khiển chi phí và phiên làm việc**: huỷ các lời gọi LLM quá hạn, ước lượng chi phí trước mỗi lời gọi để không nút nào làm vượt ngân sách, cho phép giao lại yêu cầu phê duyệt, bổ sung nhánh riêng khi phê duyệt bị từ chối, và hỗ trợ token làm mới cho phiên đăng nhập.

TÀI LIỆU THAM KHẢO

[1] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2005.

[2] Object Management Group, *OMG Unified Modeling Language (OMG UML), Version 2.5.1*, formal/17-12-05, Dec. 2017.

[3] I. Jacobson, M. Christerson, P. Jonsson, and G. Övergaard, *Object-Oriented Software Engineering: A Use Case Driven Approach*. Wokingham, U.K.: Addison-Wesley, 1992.

[4] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.

[5] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.

[6] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley, 2003.

[7] R. C. Martin, "Design principles and design patterns," Object Mentor, 2000.

[8] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA, USA: Prentice Hall, 2017.

[9] C. Larman, *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2004.

[10] B. Meyer, *Object-Oriented Software Construction*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 1997.

[11] B. H. Liskov and J. M. Wing, "A behavioral notion of subtyping," *ACM Trans. Program. Lang. Syst.*, vol. 16, no. 6, pp. 1811–1841, 1994.

[12] P. B. Kruchten, "The 4+1 view model of architecture," *IEEE Software*, vol. 12, no. 6, pp. 42–50, 1995.

[13] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2012.

[14] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, U.K.: Wiley, 2009.

[15] A. S. Rao and M. P. Georgeff, "BDI agents: From theory to practice," in *Proc. 1st Int. Conf. Multi-Agent Systems (ICMAS-95)*, San Francisco, CA, USA, 1995, pp. 312–319.

[16] Foundation for Intelligent Physical Agents, “FIPA ACL message structure specification,” SC00061G, 2002.

[17] R. G. Smith, “The contract net protocol: High-level communication and control in a distributed problem solver,” *IEEE Trans. Comput.*, vol. C-29, no. 12, pp. 1104–1113, 1980.

[18] P. Stone and M. Veloso, “Multiagent systems: A survey from a machine learning perspective,” *Autonomous Robots*, vol. 8, no. 3, pp. 345–383, 2000.

[19] Object Management Group, *Business Process Model and Notation (BPMN), Version 2.0*, formal/2011-01-03, Jan. 2011.

[20] T. Murata, “Petri nets: Properties, analysis and applications,” *Proc. IEEE*, vol. 77, no. 4, pp. 541–580, 1989.

[21] W. M. P. van der Aalst, A. H. M. ter Hofstede, B. Kiepuszewski, and A. P. Barros, “Workflow patterns,” *Distributed and Parallel Databases*, vol. 14, no. 1, pp. 5–51, 2003.

[22] M. Chen *et al.*, “Evaluating large language models trained on code,” arXiv:2107.03374, 2021.

[23] C. E. Jimenez *et al.*, “SWE-bench: Can language models resolve real-world GitHub issues?,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[24] S. Hong *et al.*, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[25] C. Qian *et al.*, “ChatDev: Communicative agents for software development,” in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024.

[26] Q. Wu *et al.*, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” arXiv:2308.08155, 2023.

[27] S. Yao *et al.*, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.

[28] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[29] J. S. Park *et al.*, “Generative agents: Interactive simulacra of human behavior,” in *Proc. 36th Annu. ACM Symp. User Interface Software and Technology (UIST)*, 2023.

[30] J. Wei *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[31] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[32] A. Vaswani *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[33] D. Harel, “Statecharts: A visual formalism for complex systems,” *Sci. Comput. Program.*, vol. 8, no. 3, pp. 231–274, 1987.

[34] K. Beck, *Test-Driven Development: By Example*. Boston, MA, USA: Addison-Wesley, 2002.

[35] T. J. McCabe, “A complexity measure,” *IEEE Trans. Softw. Eng.*, vol. SE-2, no. 4, pp. 308–320, 1976.

[36] M. H. Halstead, *Elements of Software Science*. New York, NY, USA: Elsevier North-Holland, 1977.

[37] P. Oman and J. Hagemeister, “Metrics for assessing a software system’s maintainability,” in *Proc. Conf. Software Maintenance*, Orlando, FL, USA, 1992, pp. 337–344.

[38] D. Coleman, D. Ash, B. Lowther, and P. Oman, “Using metrics to evaluate software system maintainability,” *Computer*, vol. 27, no. 8, pp. 44–49, 1994.

[39] B. W. Boehm, *Software Engineering Economics*. Englewood Cliffs, NJ, USA: Prentice Hall, 1981.

[40] A. E. Hoerl and R. W. Kennard, “Ridge regression: Biased estimation for nonorthogonal problems,” *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

[41] J. D. C. Little, “A proof for the queuing formula: L = λW,” *Oper. Res.*, vol. 9, no. 3, pp. 383–387, 1961.

[42] *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.

[43] *Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*, ISO/IEC 25010:2011, 2011.

[44] O. C. Z. Gotel and A. C. W. Finkelstein, “An analysis of the requirements traceability problem,” in *Proc. 1st Int. Conf. Requirements Engineering*, Colorado Springs, CO, USA, 1994, pp. 94–101.

[45] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[46] R. Kohavi, “A study of cross-validation and bootstrap for accuracy estimation and model selection,” in *Proc. 14th Int. Joint Conf. Artificial Intelligence (IJCAI)*, 1995, pp. 1137–1145.

[47] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” *Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[48] H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-hashing for message authentication,” RFC 2104, Feb. 1997.

[49] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, May 2015.

[50] OWASP Foundation, “OWASP Top 10:2021,” 2021.

[51] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd ed. Hoboken, NJ, USA: Wiley, 2011.

[52] M. Cohn, *Succeeding with Agile: Software Development Using Scrum*. Boston, MA, USA: Addison-Wesley, 2009.

[53] *Quality Management Systems — Fundamentals and Vocabulary*, ISO 9000:2015, 2015.

[54] M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed. Raleigh, NC, USA: Pragmatic Bookshelf, 2018.

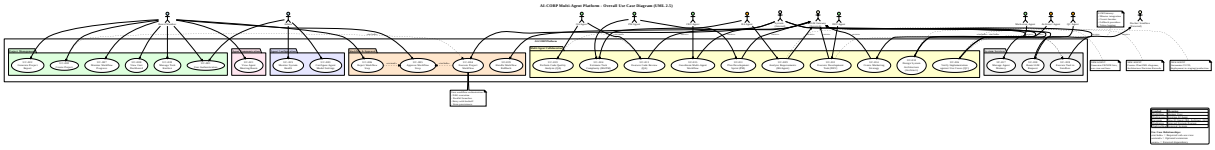
[55] D. L. Parnas, “On the criteria to be used in decomposing systems into modules,” *Commun. ACM*, vol. 15, no. 12, pp. 1053–1058, 1972.

Phụ lục A. Danh mục sơ đồ UML

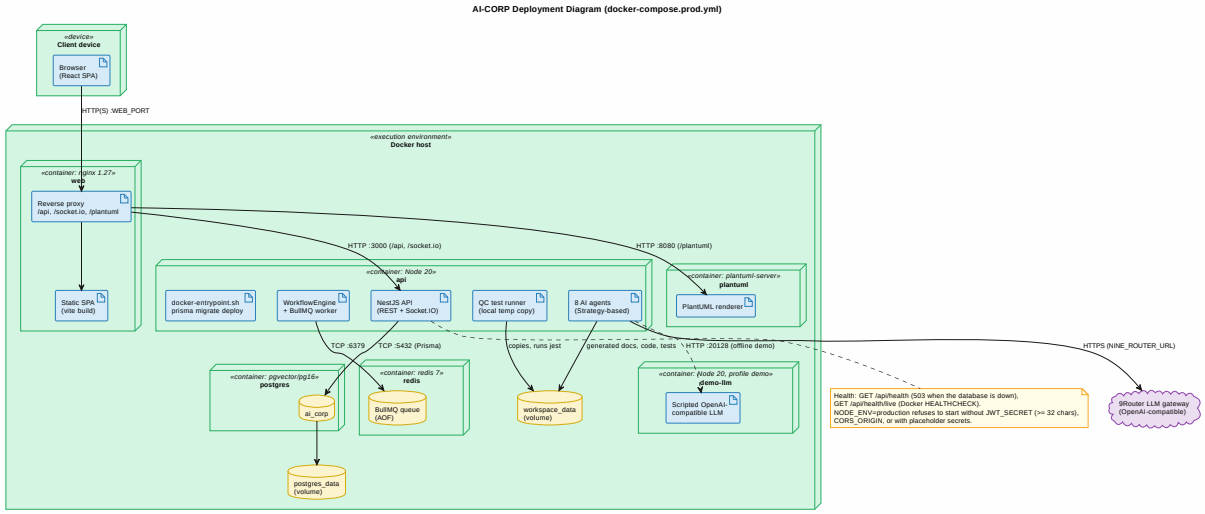
Các sơ đồ được viết bằng mã PlantUML và lưu trong thư mục `docs/diagrams/`. Lệnh `scripts/docs/render-diagrams.sh` tải PlantUML (có kiểm tra mã băm SHA-1), render toàn bộ sơ đồ sang định dạng SVG trong `docs/diagrams/rendered/`, và dừng với mã lỗi khác 0 tại sơ đồ đầu tiên có lỗi cú pháp. Danh mục đầy đủ kèm liên kết nằm trong tệp `docs/diagrams/INDEX.md`.

Góc nhìn	Số sơ đồ	Sơ đồ
Use case	4	Toàn hệ thống; quản trị viên; thành viên hội đồng; tác tử
Lớp	13	Tổng quan mô hình miền; aggregate Agent, Project, Workflow; Strategy, Observer, Repository, Factory; vòng học; cộng tác tác tử; bộ máy quy trình; bảo mật, kiểm toán, tích hợp, báo cáo; phân tích, dự báo, chất lượng mã
Tuần tự	11	SD-001 tạo dự án; SD-002 chu trình suy nghĩ–hành động; SD-003 phê duyệt; SD-004 chuỗi giao việc; SD-005 học; SD-006 ảnh chụp và khôi phục; SD-007 lập trình cặp; SD-008 quy trình con; SD-009 gọi API bằng khoá API; SD-010 gửi webhook; SD-011 xuất dự án
Hoạt động	4	AD-001 vòng đời dự án; AD-002 quy trình có nhánh song song; AD-003 ước lượng công sức và chi phí; AD-004 phân tích và sửa chất lượng mã

Góc nhìn	Số sơ đồ	Sơ đồ
Máy trạng thái	4	SM-001 tác tử; SM-002 lượt chạy quy trình; SM-003 công việc; SM-004 phê duyệt
Kiến trúc	3	Gói; thành phần; triển khai
Tổng	**39**	



Hình A.1. *Sơ đồ use case tổng thể*



Hình A.2. *Sơ đồ triển khai*

Phụ lục B. Giao diện lập trình (API)

Mọi đường dẫn đều có tiền tố `/api`. Trừ `auth/register`, `auth/login` và `health`, mọi route đều yêu cầu JWT (giao diện web) hoặc khoá API (nhóm `/v1`) và được kiểm tra quyền theo vai trò. Khi `ENABLE_SWAGGER=true`, tài liệu OpenAPI đầy đủ được cung cấp tại `/api/docs`.

Nhóm	**Route**
Xác thực	`POST auth/register`, `POST auth/login`, `GET auth/me`
Dự án	`GET/POST projects`, `GET/PATCH/DELETE projects/:id` (`PATCH` với trạng thái tạm dừng, tiếp tục hoặc huỷ tác động lên lượt chạy mới nhất), `POST projects/:id/workflow/start`
Công việc, mốc	`GET tasks`, `POST projects/:projectId/tasks`, `PATCH tasks/:id`; `GET/POST projects/:projectId/milestones`, `PATCH milestones/:id`
Sản phẩm của dự án	`GET projects/:id/files`, `GET projects/:id/files/*`, `GET projects/:id/archive` (zip), `GET projects/:id/quality-gates`
Phê duyệt	`GET approvals`, `POST approvals/:id/approve`, `POST approvals/:id/reject` (lý do bắt buộc; 409 nếu đã được trả lời)
Quy trình	`GET workflows/templates`, `POST workflows/templates/:templateId/preview`, `GET workflows/definitions[/:definitionId]`, `GET workflows/runs/:runId`, `GET workflows/runs/:runId/snapshots`, `POST workflows/runs/:runId/rollback`, `POST workflows/runs/:runId/pause`, `POST workflows/runs/:runId/resume`
Mô hình LLM	`GET models`, `GET models/available`, `PATCH models/:role` (kèm `providerId`)

Nhóm	Route
Nhà cung cấp LLM (quản trị)	`GET/POST llm-providers`, `GET llm-providers/kinds`, `PATCH/DELETE llm-providers/:id`, `POST llm-providers/:id/test`, `GET llm-providers/:id/remote-models`, `POST llm-providers/:id/models`, `PATCH/DELETE llm-providers/:id/models/:modelId`
Chi phí, báo cáo, xuất	`GET cost/summary`, `GET reports/projects/:id/cost`, `GET reports/performance`, `POST reports/weekly/run`, `GET projects/:id/export?format=json|csv|pdf`
Phân tích, dự báo, chất lượng	`GET analytics/agents`, `GET agents/:role/learning`, `GET projects/:id/estimate`, `GET analytics/estimation`, `GET projects/:id/code-quality`, `GET ooad-metrics/dashboard`, `GET ooad-metrics/project/:id`, `GET ooad-metrics/leaderboard`
Người dùng, kiểm toán	`GET users`, `GET users/roles`, `PATCH users/:id/role`, `PATCH users/:id/active`; `GET audit`, `GET audit/export`, `GET audit/anomalies`
Tích hợp (quản trị)	`GET integrations/events`, `GET/POST integrations/api-keys`, `DELETE integrations/api-keys/:id`, `GET/POST integrations/webhooks`, `DELETE integrations/webhooks/:id`, `POST integrations/webhooks/:id/test`, `GET integrations/webhooks/:id/deliveries`
API công khai (khoá API)	`GET/POST v1/projects`, `GET v1/projects/:id/status`, `GET v1/projects/:id/reports/cost`, `GET v1/projects/:id/export`, `GET/POST v1/webhooks`, `DELETE v1/webhooks/:id`
Sức khoẻ	`GET health` (503 khi cơ sở dữ liệu ngừng), `GET health/live`

WebSocket (Socket.IO, xác thực bằng JWT khi kết nối): máy khách gửi `subscribe_project` để nhận `project_event` của một dự án; máy chủ phát thêm

`system_event` (trạng thái tác tử) và `user_event`. Thông điệp `human:approval_response` bị từ chối; phê duyệt chỉ đi qua REST (mục 6.9).

Phụ lục C. Số liệu và cách tái tạo kiểm thử

Kiểm thử tự động

```
        pnpm install
        cd apps/api && npx jest                    # 50 bộ, 707 đạt, 15 bỏ qua
        cd apps/api && npm run test:cov            # độ bao phủ, thất bại nếu
dưới ngưỡng
        cd apps/web && npx vitest run && npx tsc --noEmit
```

Kiểm thử đầu-cuối

Mỗi lần chạy có script, kết quả và ảnh chụp màn hình được lưu trong một thư mục `docs/*-e2e/` của kho mã, kèm tệp README ghi lại môi trường và lệnh chạy. Script in ra một dòng cho mỗi kiểm tra và trả về mã 1 khi có kiểm tra thất bại.

Lần chạy	Script	Kiểm tra
Triển khai và trình diễn	`docs/phase7-e2e/e2e.mjs` sau `scripts/demo/seed-demo.mjs`	24
Phân tích, dự báo, chất lượng mã	`docs/phase6-e2e/e2e.mjs`	21
Bảo mật, kiểm toán, tích hợp	`docs/phase5-e2e/e2e.mjs`	44
Bộ máy quy trình (quy trình con, ghim phiên bản, ảnh chụp, khôi phục)	`docs/phase4-e2e/e2e.mjs`	theo kịch bản

Môi trường chung: PostgreSQL 16 có pgvector, Redis 7, API build ở chế độ production, LLM kịch bản `scripts/demo/demo-llm.cjs` (tương thích OpenAI, cổng 20128). Lần chạy triển khai đi qua nginx với cấu hình giống `apps/web/nginx.conf`.

LLM kịch bản

Máy chủ trả lời dựa trên vai trò và nội dung lời nhắc. Các quy tắc có ảnh hưởng đến kết quả đánh giá gồm:

Điều kiện	Câu trả lời
Tên dự án bắt đầu bằng “Legacy”	Mã lần đầu của DEV nhập hạ tầng từ miền, có khoá viết cứng, `eval` và phương thức dài; lần sửa thì sạch

Điều kiện	Câu trả lời
Tên dự án bắt đầu bằng “Unsafe”	Mã có `eval`; QA trả lời FAIL ở lần rà soát đầu, PASS ở lần hai
Mô hình của Marketing bị đánh dấu ngừng hoạt động	Lỗi 503, dùng để kiểm tra phát hiện bất thường
Mọi lời gọi	Độ trễ 20 ms + 1 ms cho mỗi 25 ký tự lời nhắc; số token theo độ dài; giá theo tên mô hình

Dữ liệu trình diễn

Script `scripts/demo/seed-demo.mjs` tạo hai tài khoản (thành viên hội đồng quản trị và người xem) cùng năm dự án đã mô tả ở mục 6.8. Script có thể chạy lặp lại: lần chạy thứ hai không tạo ra bản trùng.

Phụ lục D. Số liệu hiệu năng

Kiểm thử tải

Script `scripts/perf/load-test.mjs` (không phụ thuộc thư viện), ngân sách trong `scripts/perf/budgets.json`:

```
        node scripts/perf/load-test.mjs --api http://localhost:3000/api --users
20 --duration 30 --create-every 0
        node scripts/perf/load-test.mjs --users 20 --duration 30 --create-every
10   # có quy trình đang chạy
        node scripts/perf/load-test.mjs --users 50 --duration 30 --create-every
25   # bão hoà
```

Kết quả chi tiết với 20 người dùng chỉ đọc (30 s, 18 154 yêu cầu, 604,8 yêu cầu/giây, 0% lỗi; cơ sở dữ liệu bắt đầu từ 5 dự án trình diễn):

Endpoint	Số yêu cầu	p50 (ms)	p95 (ms)	p99 (ms)	Ngân sách p50/p95/p99
`GET /analytics/agents`	1 832	30	44	52	300/1500/3000
`GET /health`	911	54	84	97	100/500/1000
`GET /projects`	4 472	28	42	51	100/500/1000
`GET /projects/:id`	3 674	31	46	54	100/500/1000
`GET /projects/:id/code-quality`	1 794	20	33	44	300/1500/3000
`GET /projects/:id/estimate`	1 838	47	66	78	300/1500/3000
`GET /reports/projects/:id/cost`	1 751	36	51	59	100/500/1000
`GET /tasks?projectId`	1 882	27	41	51	100/500/1000

Ba lỗi hiệu năng tìm được ở lần đo trước (cơ sở dữ liệu khoảng 120 dự án) và hiệu quả của bản sửa (cùng kịch bản 50 người dùng có tạo dự án):

Lỗi	Cách sửa	Kết quả
Endpoint chất lượng mã phân tích lại mọi tệp ở mỗi yêu cầu	Lưu kết quả theo mã băm nội dung tệp và theo dự án trong 30 s; DEV xoá bộ đệm khi ghi lại mã; đọc tệp song song	p50 3 116 → 196 ms
Danh sách dự án tải JSON ngữ cảnh của lượt chạy rồi bỏ đi	Repository chỉ chọn các cột DTO cần	46 → 9 ms (1 người dùng)

Lỗi	Cách sửa	Kết quả
Chi tiết dự án chạy bốn truy vấn độc lập tuần tự	Chạy song song	16 → 9 ms (1 người dùng)

Trong lần đo đó, sau khi sửa, thông lượng ở kịch bản bão hoà tăng từ 98,8 lên 178,3 yêu cầu/giây.

Thời gian theo bước tác tử

Bảng theo vai trò nằm ở mục 6.4. Dữ liệu gốc là trường `durationMs` của các bước trong lượt chạy, đọc qua `GET /api/workflows/runs/:runId`; các lượt chạy của lần kiểm thử triển khai được lưu trong `docs/phase7-e2e/out/`.

Phụ lục E. Hướng dẫn sử dụng

Cài đặt bằng Docker

```
        cp .env.production.example .env.production
        # Điền POSTGRES_PASSWORD, JWT_SECRET (openssl rand -hex 32, tối thiểu
32 ký tự),
        # CORS_ORIGIN (địa chỉ web), NINE_ROUTER_URL và NINE_ROUTER_API_KEY
(cổng LLM),
        # SEED_ADMIN_EMAIL và SEED_ADMIN_PASSWORD (tối thiểu 12 ký tự),
LLM_SECRET_KEY (mã hoá khoá nhà cung cấp)
        # Tuỳ chọn: SIGNUP_ROLE (mặc định viewer), ALLOW_REGISTRATION,
JWT_EXPIRATION (giây, mặc định 8 giờ)
        docker compose -f docker-compose.prod.yml --env-file .env.production up
-d --build
```

Giao diện web ở `http://localhost:8080` (đổi bằng `WEB_PORT`). Lần khởi động đầu chạy migration và tạo tài khoản quản trị; sau đó đặt `SEED_ON_START=false`. API từ chối khởi động nếu còn giá trị mẫu (`CHANGE_ME`), thiếu `JWT_SECRET` hoặc `CORS_ORIGIN`, hoặc khoá JWT ngắn hơn 32 ký tự, và in đủ danh sách lỗi. Thêm `--profile demo` để chạy kèm LLM kịch bản phục vụ trình diễn. Lần khởi động đầu cũng nhập cổng LLM trong `NINE_ROUTER_URL` thành nhà cung cấp đầu tiên; các nhà cung cấp khác được thêm trên trang *LLM Providers*.

Vai trò người dùng

Vai trò	Có thể
Quản trị viên (`admin`)	Mọi thao tác, gồm quản lý người dùng và vai trò, khoá API, webhook, nhà cung cấp và mô hình LLM, xem mọi phê duyệt đang chờ
Thành viên hội đồng (`board_member`)	Tạo và quản lý dự án của mình, duyệt hoặc từ chối phê duyệt, xem báo cáo và phân tích
Người xem (`viewer`)	Chỉ xem; giao diện ẩn các nút tạo, sửa, duyệt và API trả 403 nếu gọi trực tiếp. Tài khoản tự đăng ký có vai trò này cho đến khi quản trị viên nâng quyền

Các thao tác chính

Giao diện sử dụng nhãn tiếng Anh; tên màn hình được giữ nguyên văn và in nghiêng.

1. **Tạo dự án:** *Dashboard* → *New Project*, nhập tên, mô tả yêu cầu, ngân sách, chọn mẫu quy trình (ví dụ dự án phần mềm OOAD hoặc chỉ phân tích và thiết

kế) và tham số của mẫu. Quy trình bắt đầu ngay.

2. **Theo dõi:** *Virtual Office* cho thấy trạng thái tám tác tử theo thời gian thực, và *Meeting Room* hiển thị trao đổi giữa các tác tử. Trang chi tiết dự án có tab *Workflow Runs* với dòng thời gian, ảnh chụp và nút khôi phục.
3. **Điều khiển và phê duyệt:** đầu trang chi tiết dự án có các nút tạm dừng, tiếp tục, huỷ và chỉnh ngân sách (không tiếp tục được khi đang chờ phê duyệt hoặc đã hết ngân sách). Khi quy trình đến nút phê duyệt, chuông thông báo hiện cho thành viên hội đồng; hộp thoại phê duyệt có tóm tắt sản phẩm, và từ chối thì phải nêu lý do.
4. **Xem sản phẩm:** tab *Artifacts* có tóm tắt OOAD, kết quả cổng chất lượng, cây tệp, trình xem Markdown, mã và sơ đồ, và nút tải zip.
5. **Báo cáo và dự báo:** tab *Reports* có chi phí theo vai trò, dự báo thời gian và chi phí, điểm chất lượng mã; xuất JSON, CSV hoặc PDF.
6. **Phân tích tác tử và mô hình:** *Agent Analytics* so sánh hiệu năng các vai trò và cảnh báo bất thường; *LLM Providers* thêm nhà cung cấp (mẫu có sẵn cho 9Router, OpenAI, OpenRouter, Groq, DeepSeek, Gemini, Ollama, Anthropic), kiểm tra kết nối và lấy mô hình; *Agent Models* chọn nhà cung cấp và mô hình cho từng vai trò.
7. **Quản trị:** *Users & Roles* (vai trò, kích hoạt), *Audit Log* (lọc, xuất CSV, bất thường), *Integrations* (khoá API, webhook, lịch sử gửi).